PRACTICAL REPORT

AWS CLOUD & DATA ENGINEERING

Amazon S3 • AWS Glue • Linux • SQL

SUBMITTED BY

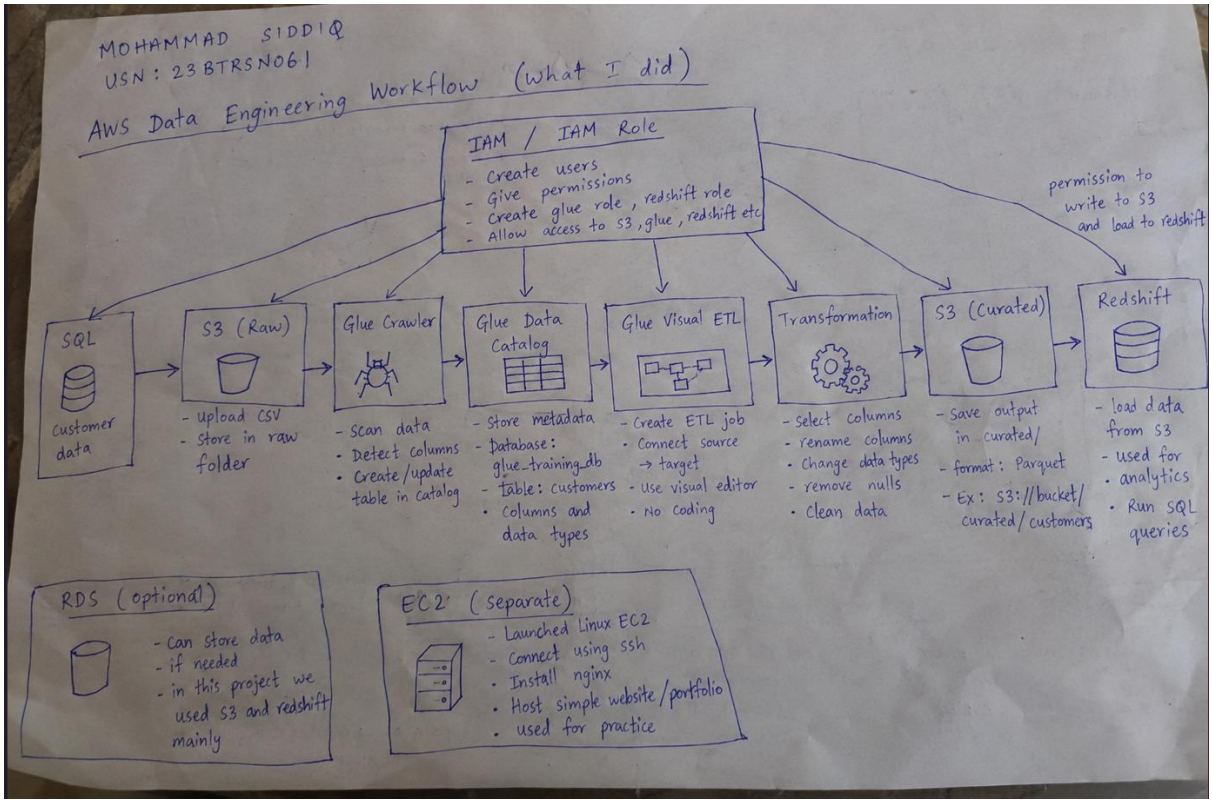
Mohammad Siddiq Shaik

ROLL NUMBER	**23BTRSN061**
COLLEGE NAME	JAIN (Deemed-to-be University), Bengaluru, Karnataka
PROGRAMME	B.Tech — Computer Science Engineering
BATCH	2023–2027
SUBMISSION DATE	11 October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Academic Practical Documentation

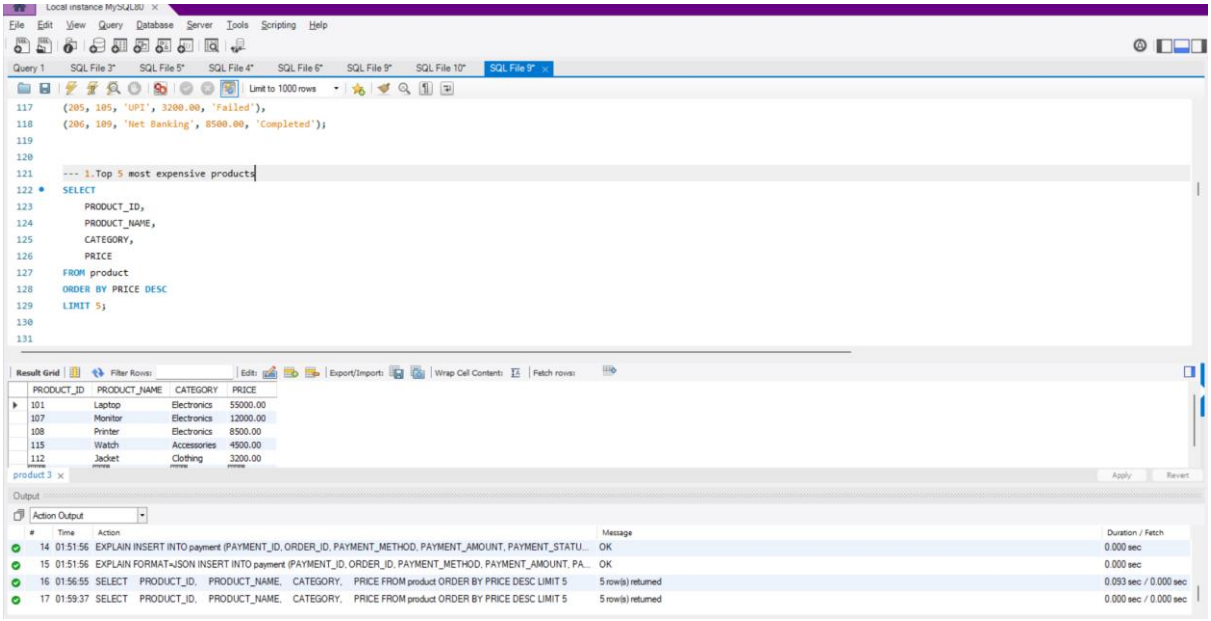
WHAT I DID SO FAR!



SQL Practice(MY SQL WORKBENCH)

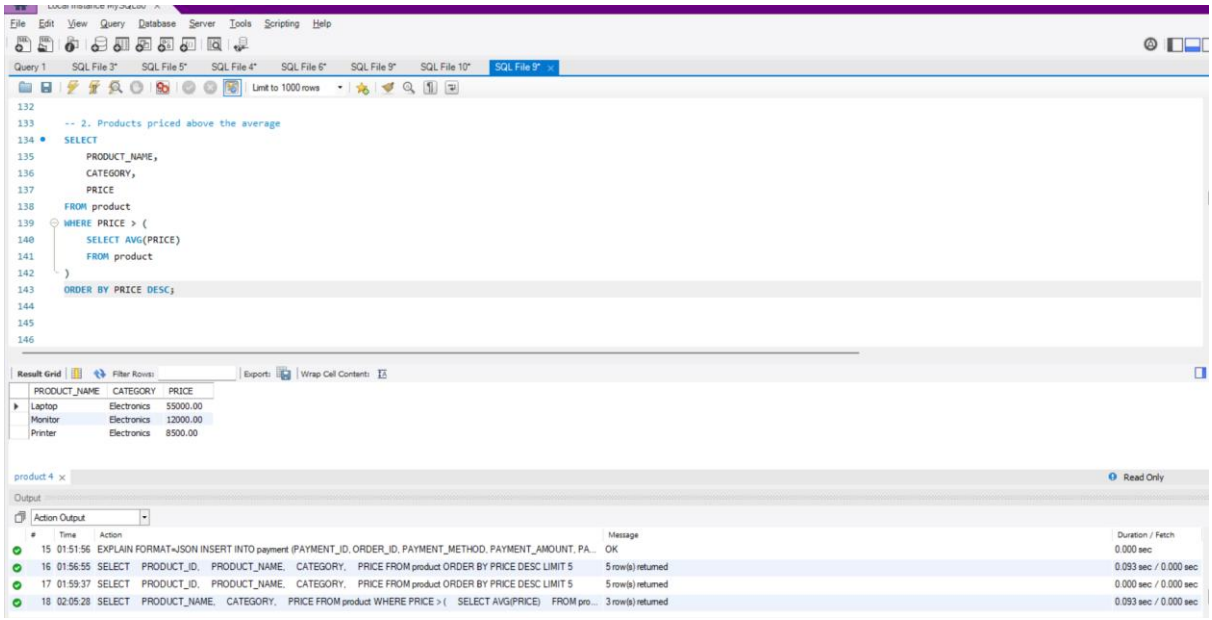
1. Top 5 most expensive products

Purpose: Identifies the five most expensive products in the catalog.



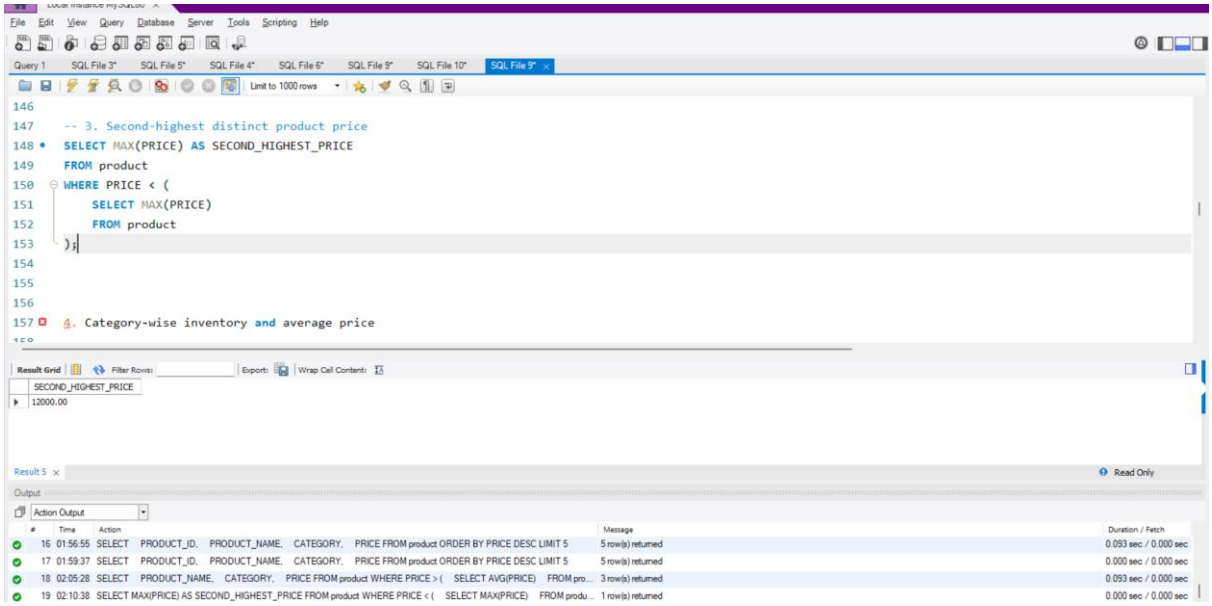
2. Products priced above the average

Purpose: Finds products priced higher than the average product price.



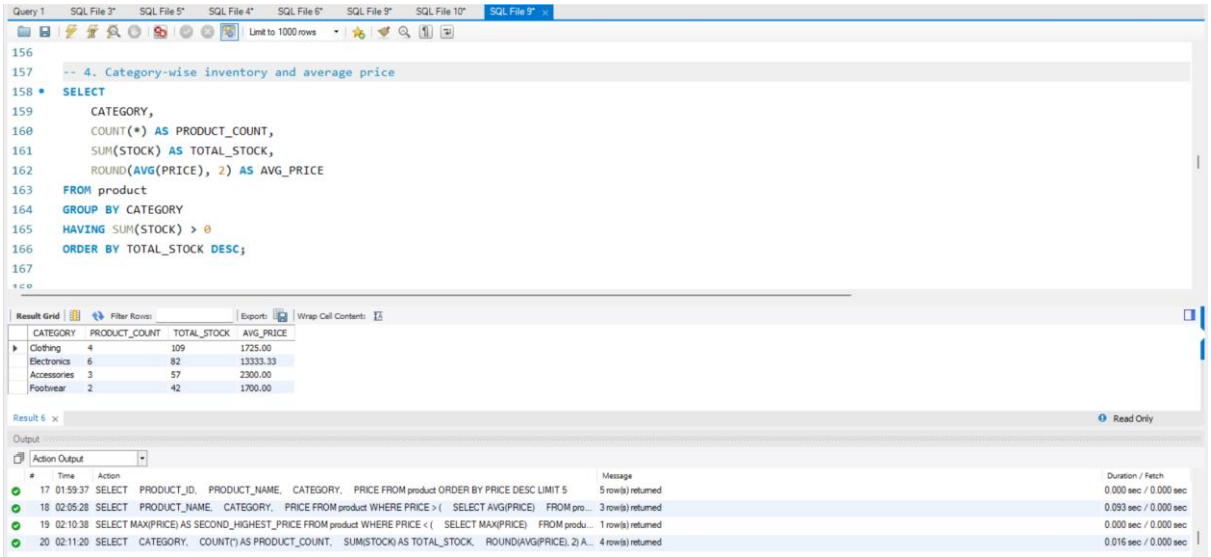
3. Second-highest distinct product price

Purpose: Finds the second-highest distinct price, even if multiple products share the highest price.



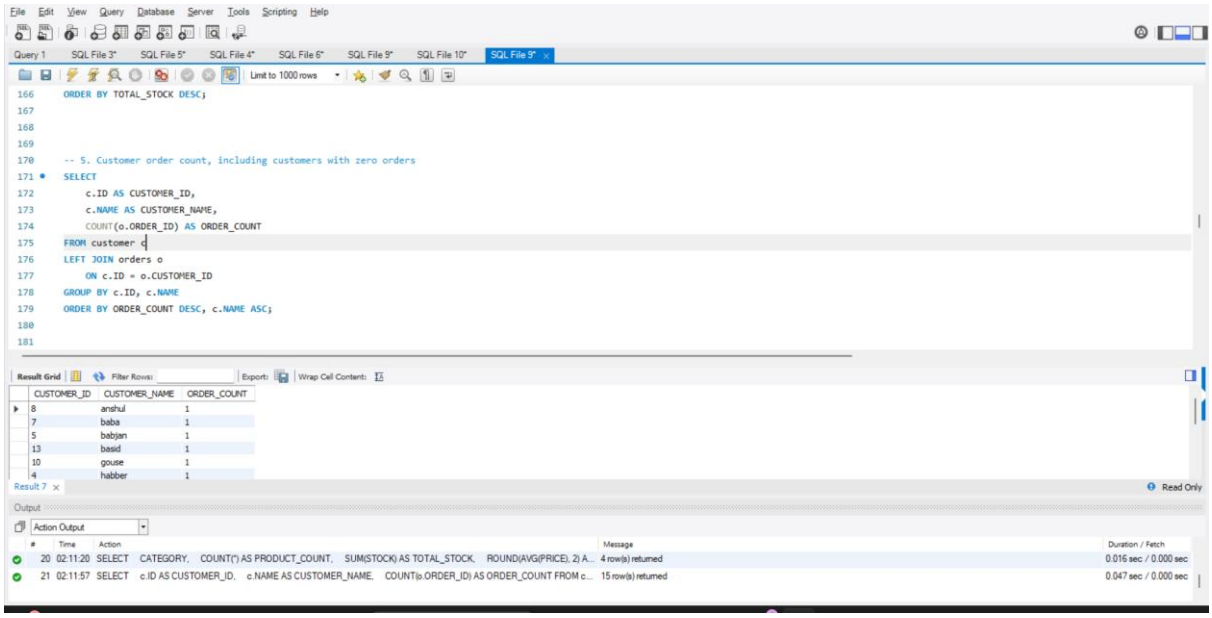
4. Category-wise inventory and average price

Purpose: Compares product availability and average pricing across categories.



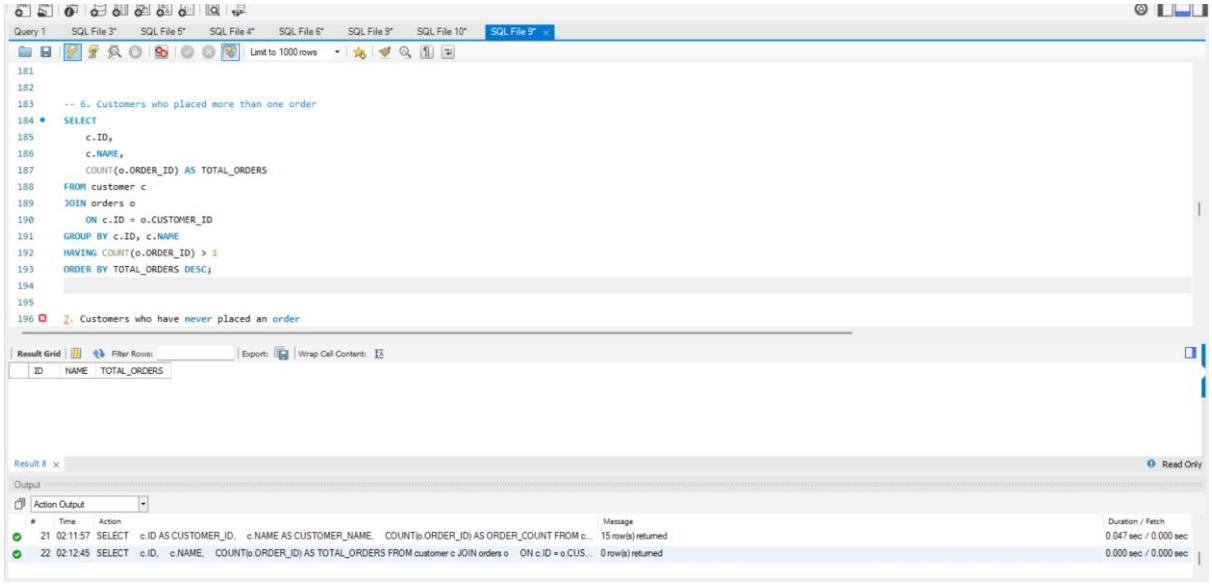
5. Customer order count, including customers with zero orders

Purpose: Identifies frequent buyers while retaining customers who have never placed an order.



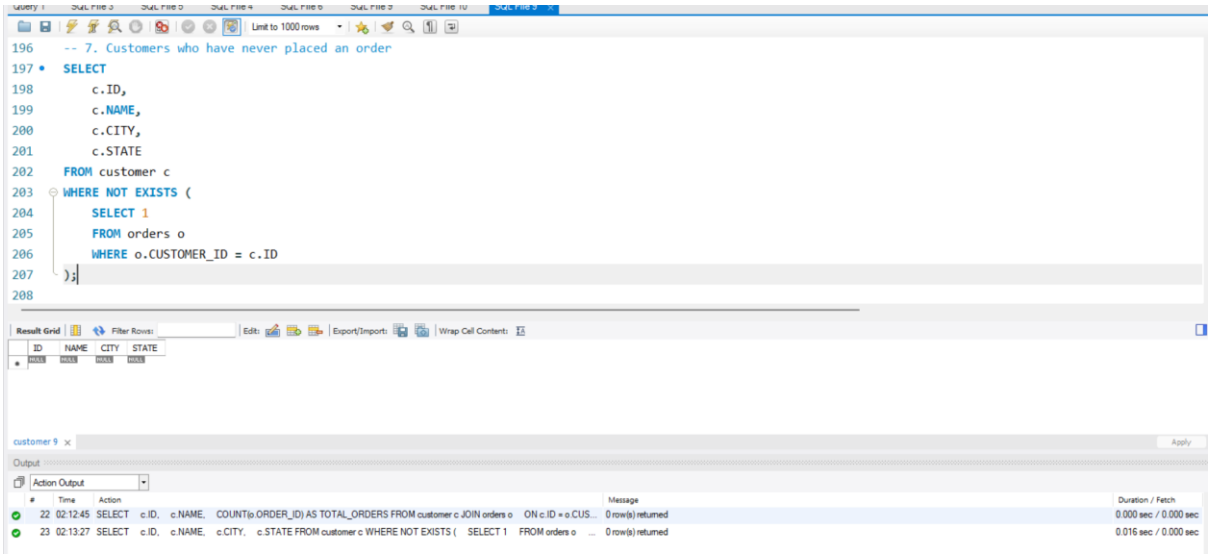
6. Customers who placed more than one order

Purpose: Identifies repeat customers for customer-retention analysis.



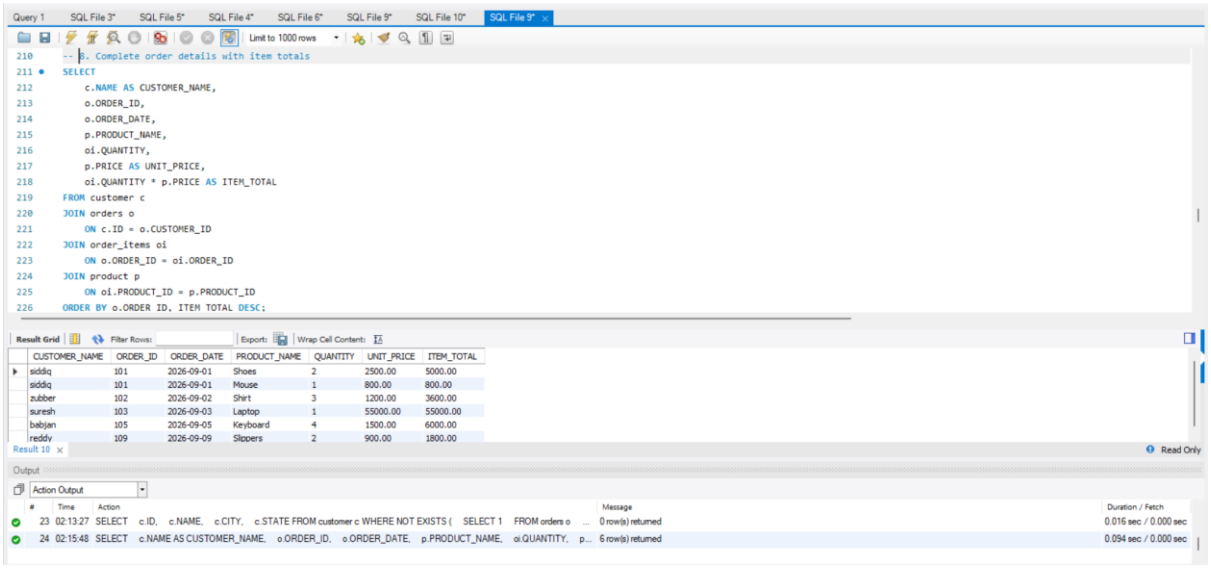
7. Customers who have never placed an order

Purpose: Finds customers who may need engagement or promotional offers. NOT EXISTS also avoids the NULL pitfalls that can affect NOT IN.



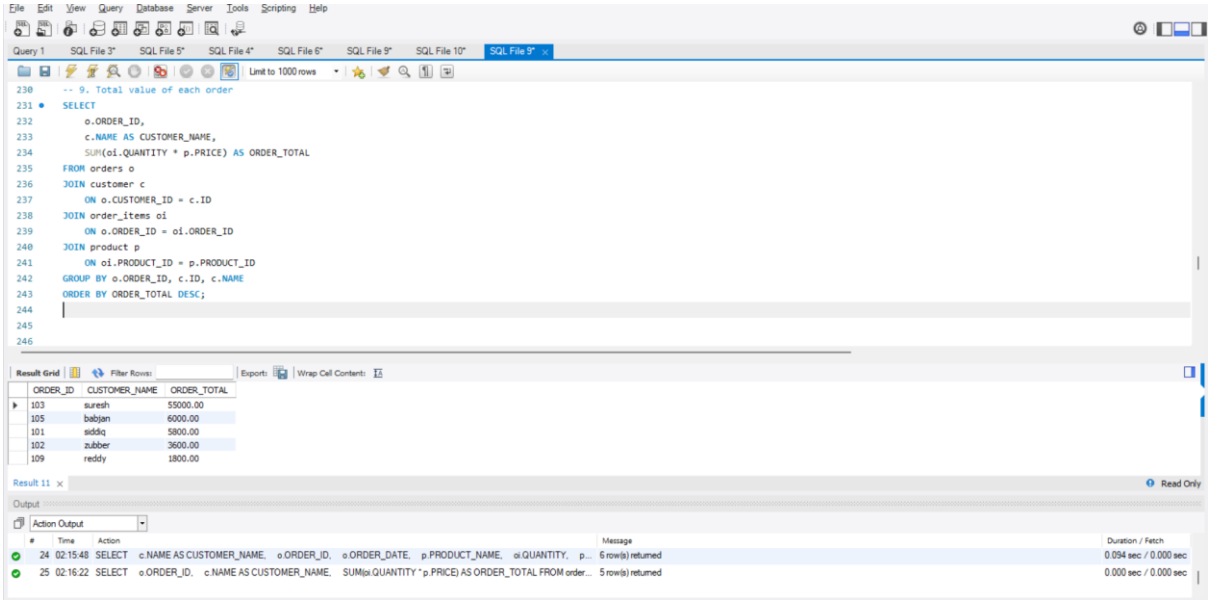
8. Complete order details with item totals

Purpose: Shows which customers bought which products and the calculated value of each order item.



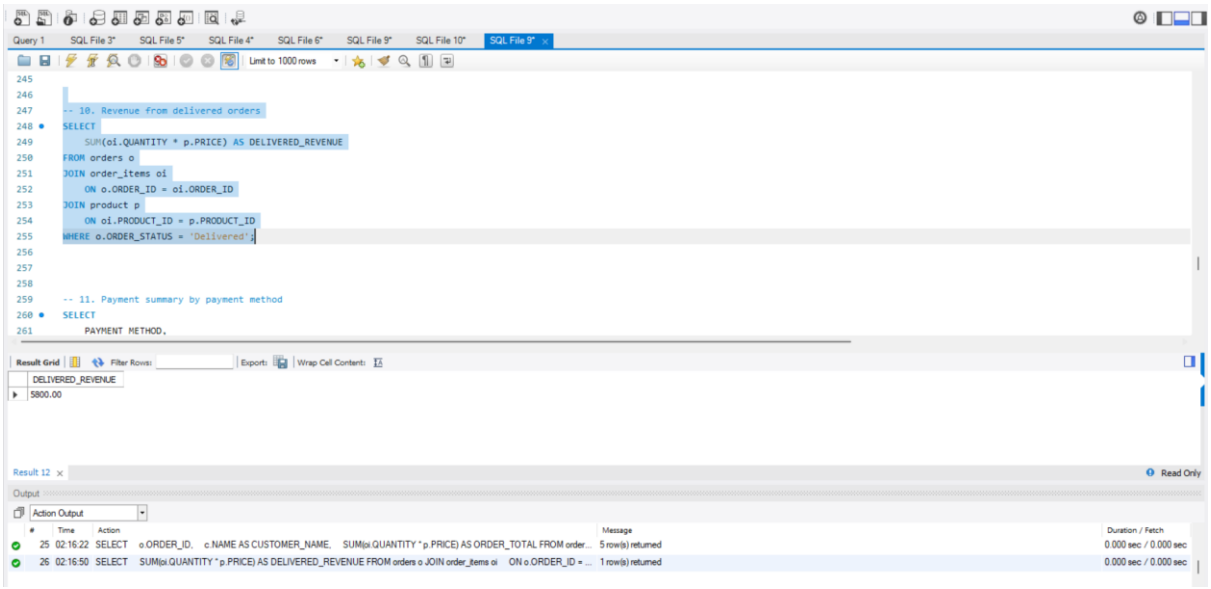
9. Total value of each order

Purpose: Calculates the total merchandise value of each order from its items.



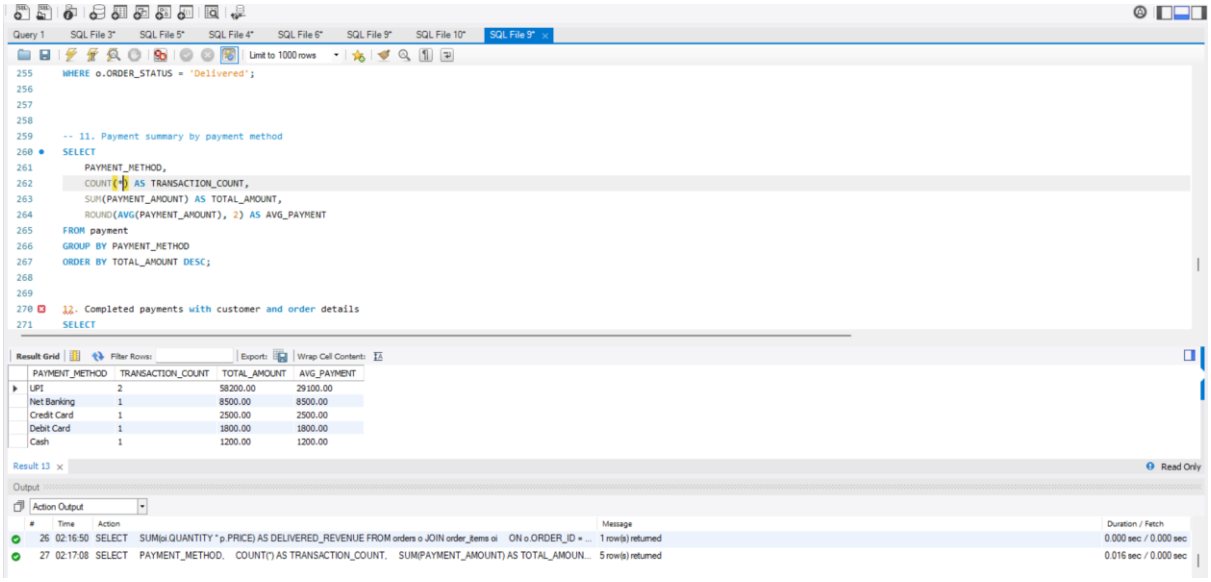
10. Revenue from delivered orders

Purpose: Calculates the merchandise value associated with delivered orders.



11. Payment summary by payment method

Purpose: Compares payment methods by transaction count, total payment amount, and average payment.



```
WHERE o.ORDER_STATUS = 'Delivered';



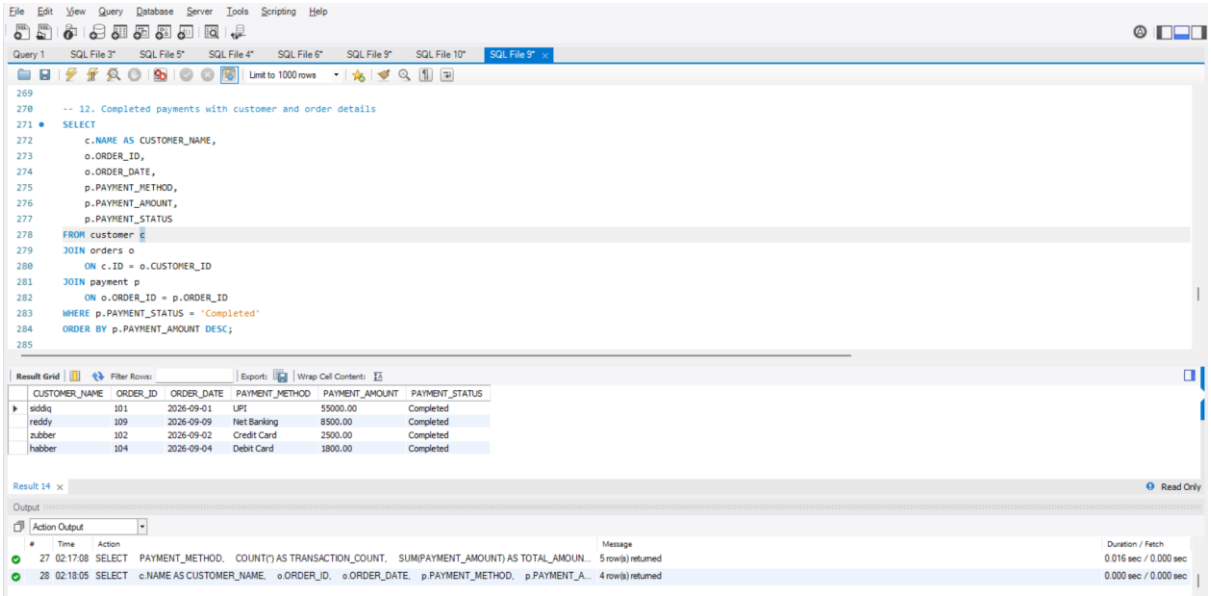
-- 11. Payment summary by payment method
SELECT
    PAYMENT_METHOD,
    COUNT(*) AS TRANSACTION_COUNT,
    SUM(PAYMENT_AMOUNT) AS TOTAL_AMOUNT,
    ROUND(AVG(PAYMENT_AMOUNT), 2) AS AVG_PAYMENT
FROM payment
GROUP BY PAYMENT_METHOD
ORDER BY TOTAL_AMOUNT DESC;


-- 12. Completed payments with customer and order details
SELECT
```

PAYMENT_METHOD	TRANSACTION_COUNT	TOTAL_AMOUNT	AVG_PAYMENT
UPI	2	58200.00	29100.00
Net Banking	1	8500.00	8500.00
Credit Card	1	2500.00	2500.00
Debit Card	1	1800.00	1800.00
Cash	1	1200.00	1200.00

12. Completed payments with customer and order details

Purpose: Lists orders associated with completed payment records.

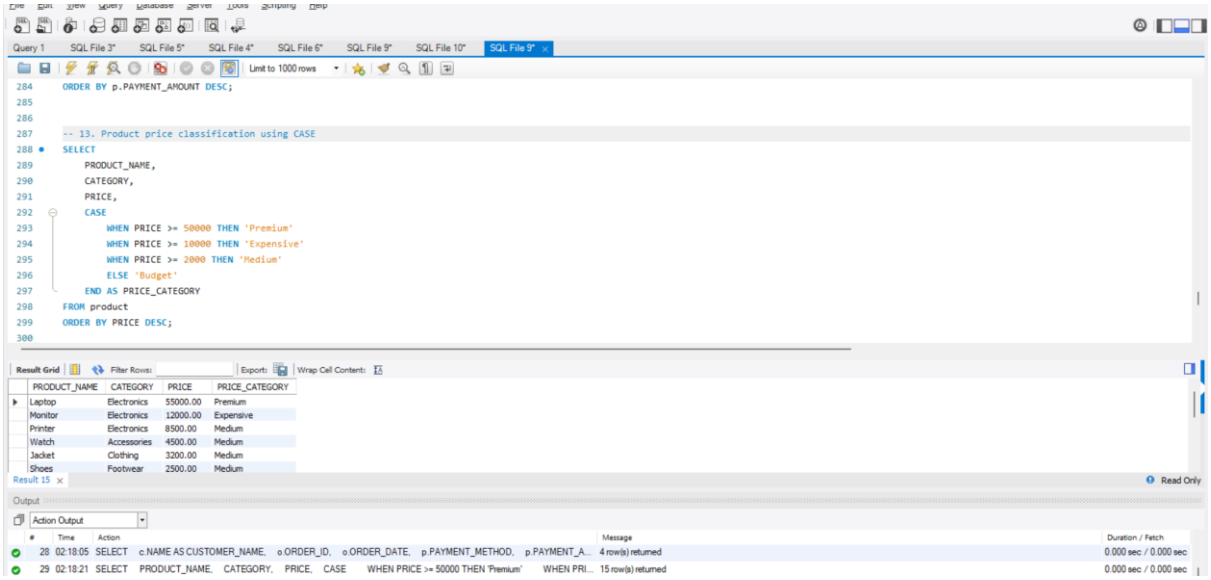


```
-- 12. Completed payments with customer and order details
SELECT
    c.NAME AS CUSTOMER_NAME,
    o.ORDER_ID,
    o.ORDER_DATE,
    p.PAYMENT_METHOD,
    p.PAYMENT_AMOUNT,
    p.PAYMENT_STATUS
FROM customer c
JOIN orders o
    ON c.ID = o.CUSTOMER_ID
JOIN payment p
    ON o.ORDER_ID = p.ORDER_ID
WHERE p.PAYMENT_STATUS = 'Completed'
ORDER BY p.PAYMENT_AMOUNT DESC;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PAYMENT_METHOD	PAYMENT_AMOUNT	PAYMENT_STATUS
siddiq	101	2026-09-01	UPI	55000.00	Completed
reddy	109	2026-09-09	Net Banking	8500.00	Completed
zubber	102	2026-09-02	Credit Card	2500.00	Completed
habber	104	2026-09-04	Debit Card	1800.00	Completed

13. Product price classification using CASE

Purpose: Classifies products into pricing segments for catalog analysis.



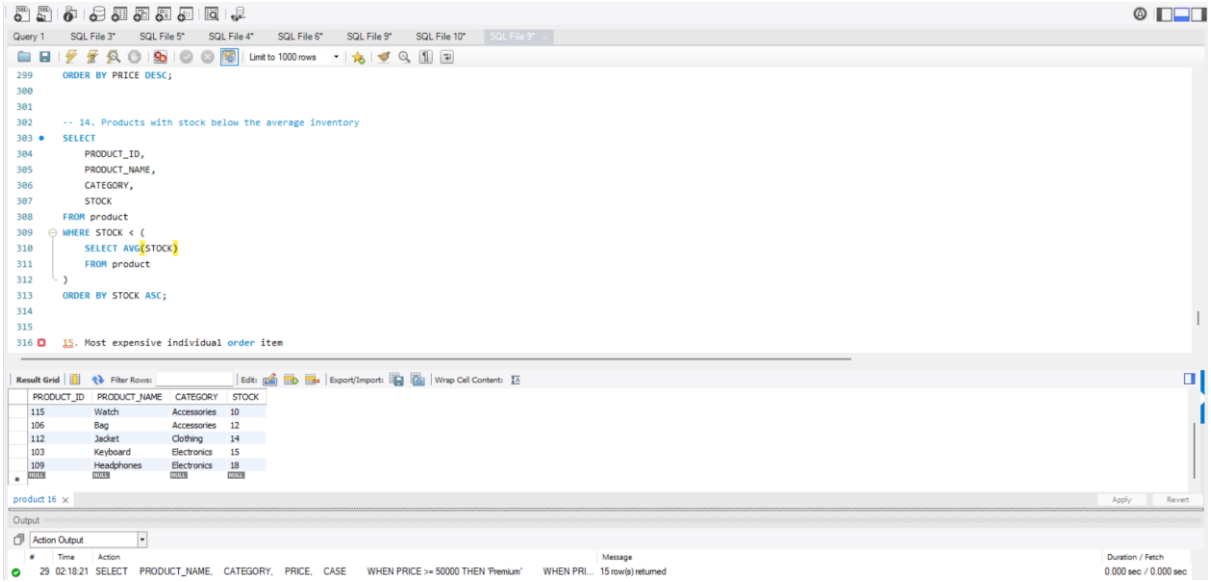
```
ORDER BY p.PAYMENT_AMOUNT DESC;


-- 13. Product price classification using CASE
SELECT
    PRODUCT_NAME,
    CATEGORY,
    PRICE,
    CASE
        WHEN PRICE >= 50000 THEN 'Premium'
        WHEN PRICE >= 10000 THEN 'Expensive'
        WHEN PRICE >= 2000 THEN 'Medium'
        ELSE 'Budget'
    END AS PRICE_CATEGORY
FROM product
ORDER BY PRICE DESC;
```

PRODUCT_NAME	CATEGORY	PRICE	PRICE_CATEGORY
Laptop	Electronics	55000.00	Premium
Monitor	Electronics	12000.00	Expensive
Printer	Electronics	8500.00	Medium
Watch	Accessories	4500.00	Medium
Jacket	Clothing	3200.00	Medium
Shoes	Footwear	2500.00	Medium

14. Products with stock below the average inventory

Purpose: Highlights products with comparatively low inventory that may require restocking.



```
ORDER BY PRICE DESC;

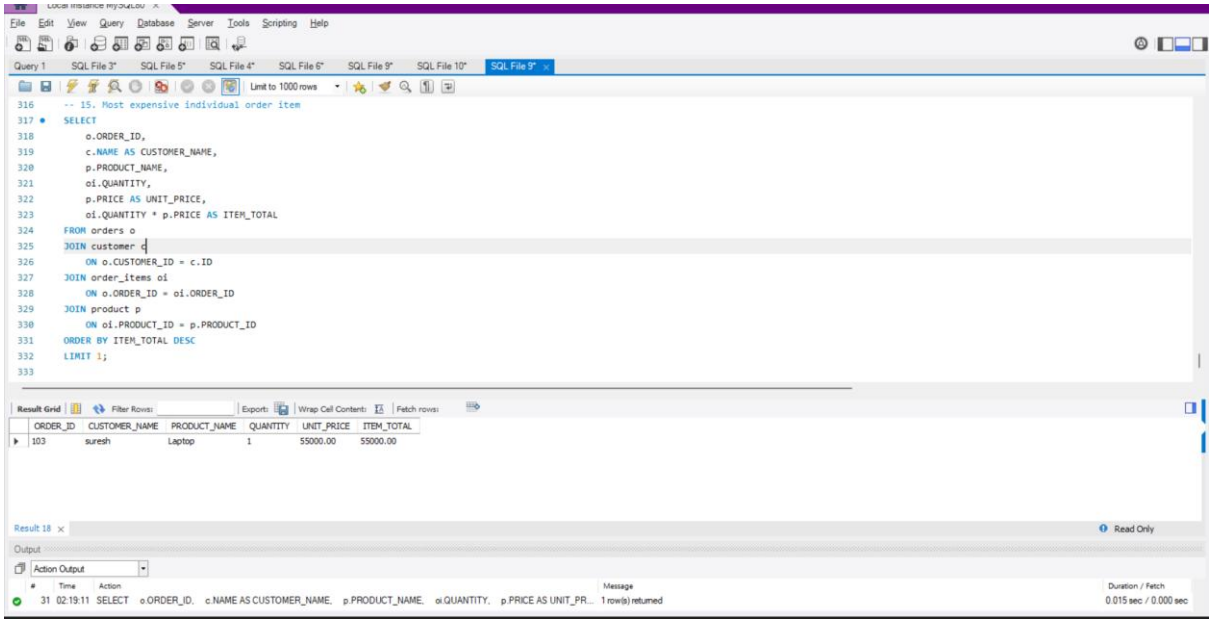

-- 14. Products with stock below the average inventory
SELECT
    PRODUCT_ID,
    PRODUCT_NAME,
    CATEGORY,
    STOCK
FROM product
WHERE STOCK < (
    SELECT AVG(STOCK)
    FROM product
)
ORDER BY STOCK ASC;


15. Most expensive individual order item
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY	STOCK
115	Watch	Accessories	10
106	Bag	Accessories	12
112	Jacket	Clothing	14
103	Keyboard	Electronics	15
109	Headphones	Electronics	18

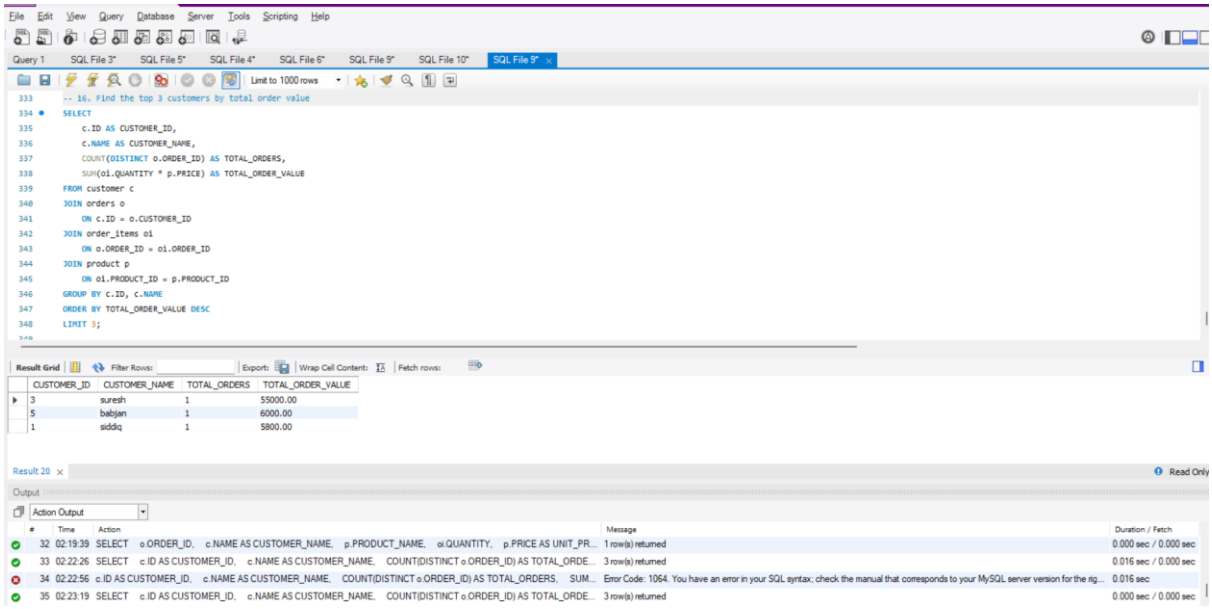
15. Most expensive individual order item

Purpose: Identifies the order containing the highest-value individual product line.



16. Find the top 3 customers by total order value

Purpose: Identify the three customers with the highest total value of purchased products.



Linux Practice

1. Basic Linux commands

No.	Command	Use
1	pwd	Displays the current working directory.
2	whoami	Displays the current logged-in username.
3	date	Displays the system date and time.
4	uname -a	Displays Linux kernel and system information.
5	ls -lah	Lists files, including hidden files, with details.
6	ls	Lists files and directories in the current location.
7	cd linux_practice	Changes the current directory to linux_practice.
8	cd ..	Moves to the parent directory.
9	cd ~	Navigates to the user's home directory.

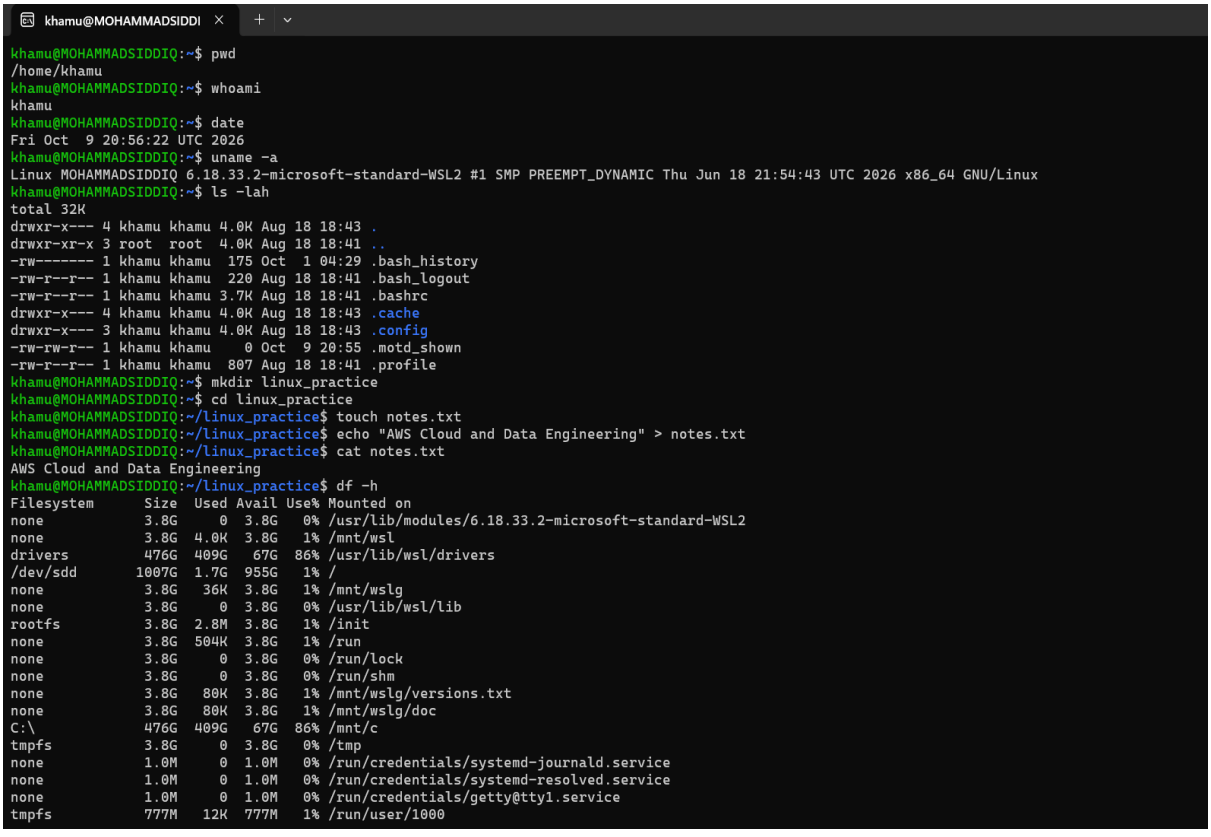
2. Directory and file management

No.	Command	Use
10	mkdir linux_practice	Creates a directory named linux_practice.
11	mkdir -p ~/linux_practice/portfolio	Creates the portfolio directory and any missing parent directories.
12	touch notes.txt	Creates an empty file named notes.txt.
13	touch index.html style.css about.html	Creates three website files.
14	nano index.html	Opens the HTML file for editing in Nano.
15	nano style.css	Opens the CSS file for editing in Nano.
16	nano about.html	Opens the About page for editing in Nano.
17	echo "AWS Cloud and Data Engineering" > notes.txt	Writes text to notes.txt, replacing existing content.
18	cat notes.txt	Displays the contents of notes.txt.

3. Website development and verification

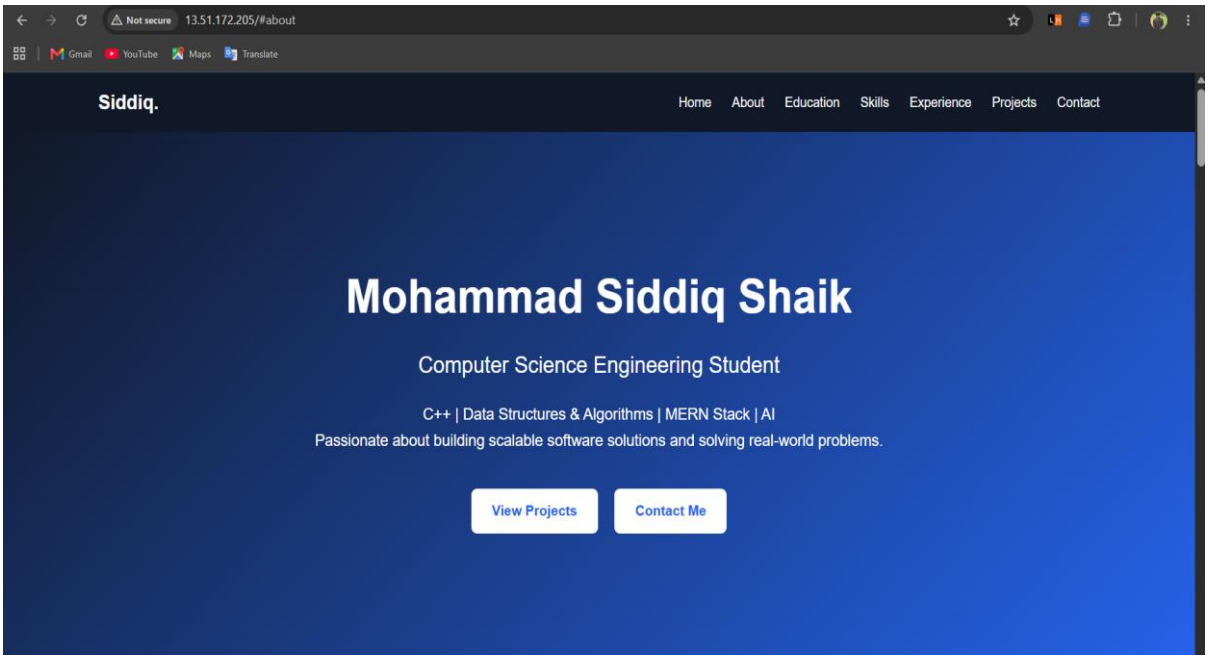
No.	Command or action	Use (one line)
23	mkdir -p ~/linux_practice/portfolio	Creates the website project directory.
24	cd ~/linux_practice/portfolio	Opens the website project directory.
25	touch index.html style.css about.html	Creates the homepage, stylesheet, and About page.
26	nano index.html	Creates or edits the website homepage HTML.
27	nano style.css	Creates or edits the website styling.
28	nano about.html	Creates or edits the About page HTML.
29	ls -lah	Verifies files and directories, including hidden files.
30	cat notes.txt	Verifies the text saved in the notes file.

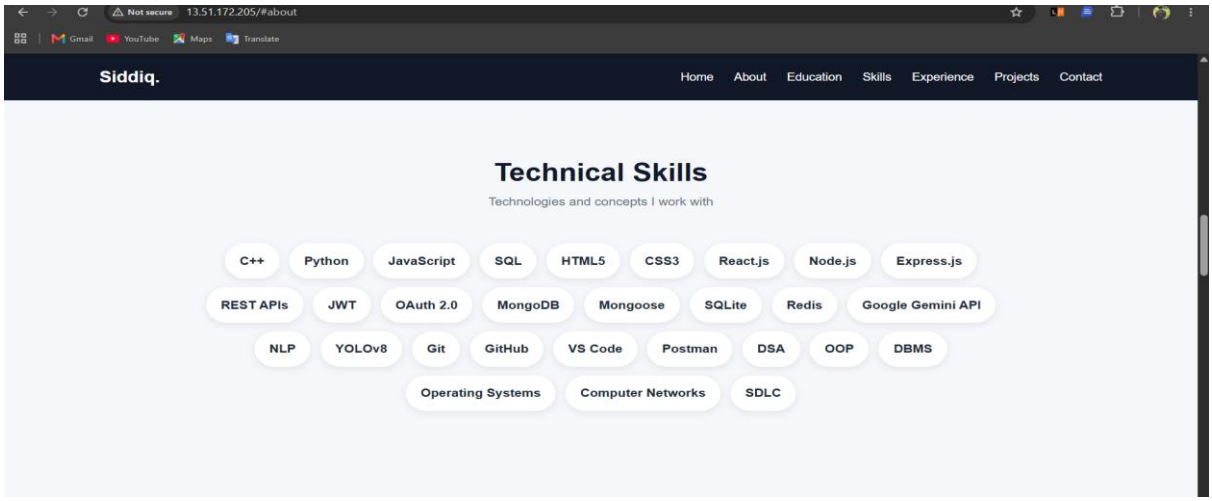
TERMINAL(UBUNTU)

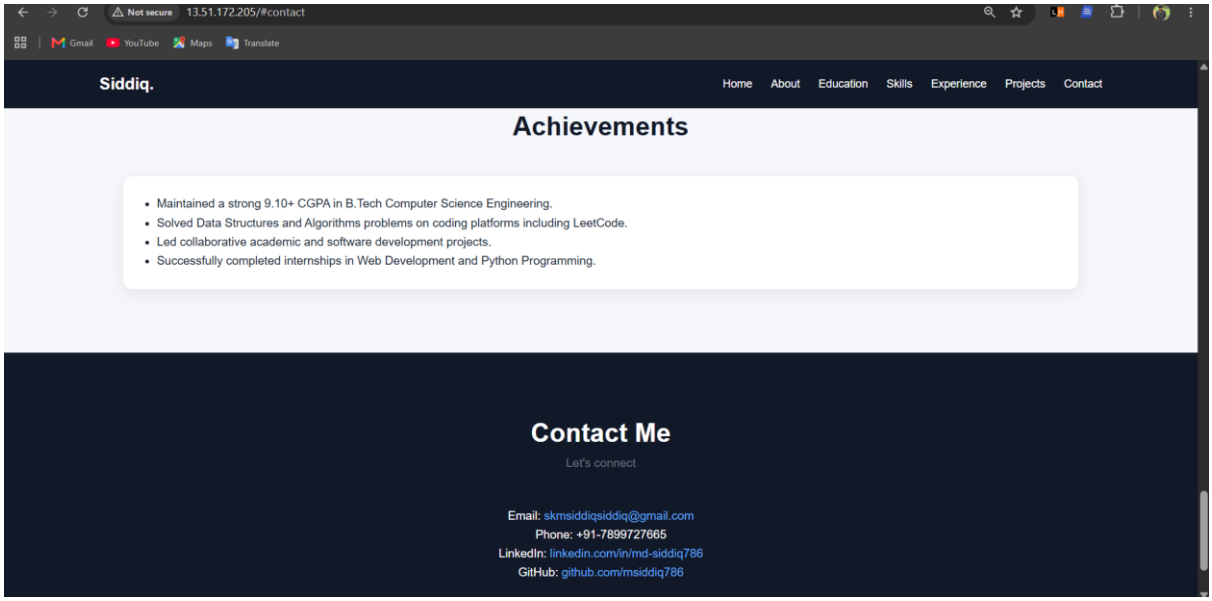


```
khamu@MOHAMMADSIDDI
tmpfs           777M   12K  777M   1% /run/user/1000
khamu@MOHAMMADSIDDIQ:~/linux_practice$ systemctl status nginx
Unit nginx.service could not be found.
khamu@MOHAMMADSIDDIQ:~/linux_practice$ ls
notes.txt
khamu@MOHAMMADSIDDIQ:~/linux_practice$ cd ..
khamu@MOHAMMADSIDDIQ:~$ ls
linux_practice
khamu@MOHAMMADSIDDIQ:~$ cd ~
khamu@MOHAMMADSIDDIQ:~$ ls -lah
total 36K
drwxr-x--- 5 khamu khamu 4.0K Oct  9 20:56 .
drwxr-xr-x 3 root  root  4.0K Aug 18 18:41 ..
-rw------- 1 khamu khamu  175 Oct  1 04:29 .bash_history
-rw-r--r-- 1 khamu khamu  220 Aug 18 18:41 .bash_logout
-rw-r--r-- 1 khamu khamu 3.7K Aug 18 18:41 .bashrc
drwxr-x--- 4 khamu khamu 4.0K Aug 18 18:43 .cache
drwxr-x--- 3 khamu khamu 4.0K Aug 18 18:43 .config
-rw-rw-r-- 1 khamu khamu    0 Oct  9 20:55 .motd_shown
-rw-r--r-- 1 khamu khamu  807 Aug 18 18:41 .profile
drwxr-xr-x 2 khamu khamu 4.0K Oct  9 20:57 linux_practice
khamu@MOHAMMADSIDDIQ:~$ cd linux_practice
khamu@MOHAMMADSIDDIQ:~/linux_practice$ mkdir -p ~/linux_practice/portfolio
khamu@MOHAMMADSIDDIQ:~/linux_practice$ cd ~/linux_practice/portfolio
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ pwd
/home/khamu/linux_practice/portfolio
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ touch index.html style.css about.html
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ nano index.html
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ nano style.css
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ nano about.html
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ python3 --version
Python 3.14.4
khamu@MOHAMMADSIDDIQ:~/linux_practice/portfolio$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [09/Oct/2026 21:04:57] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 21:04:58] "GET /style.css HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 21:04:58] code 404, message File not found
127.0.0.1 - - [09/Oct/2026 21:04:58] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [09/Oct/2026 21:05:00] "GET /about.html HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 21:05:01] "GET /index.html HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 21:05:04] "GET /about.html HTTP/1.1" 304 -
```

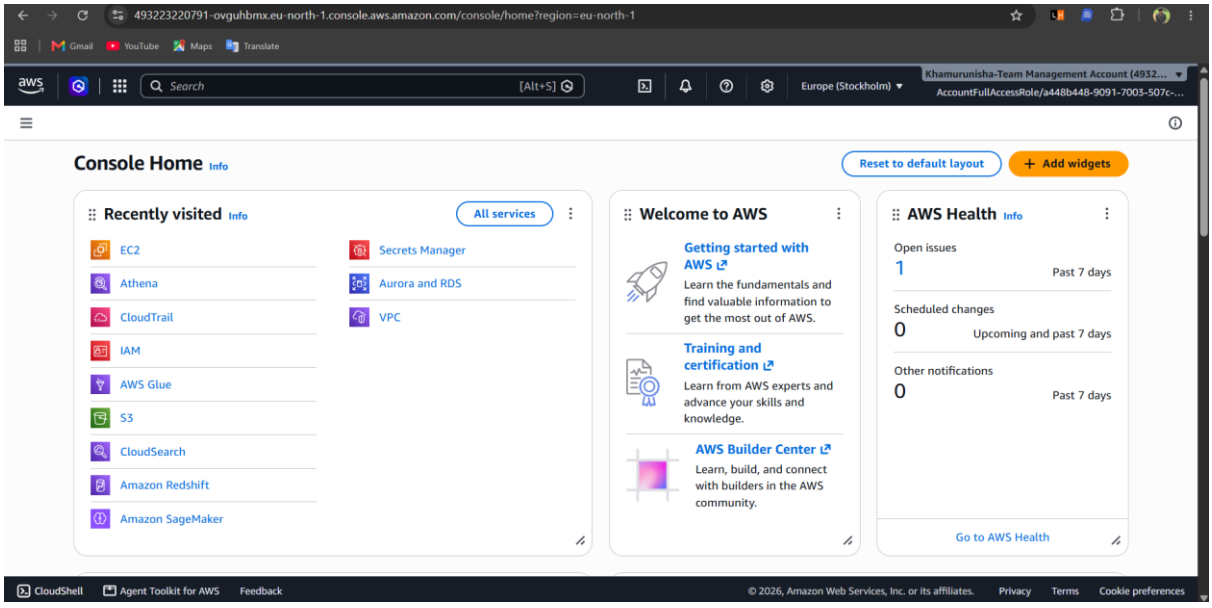
. OUTPUT





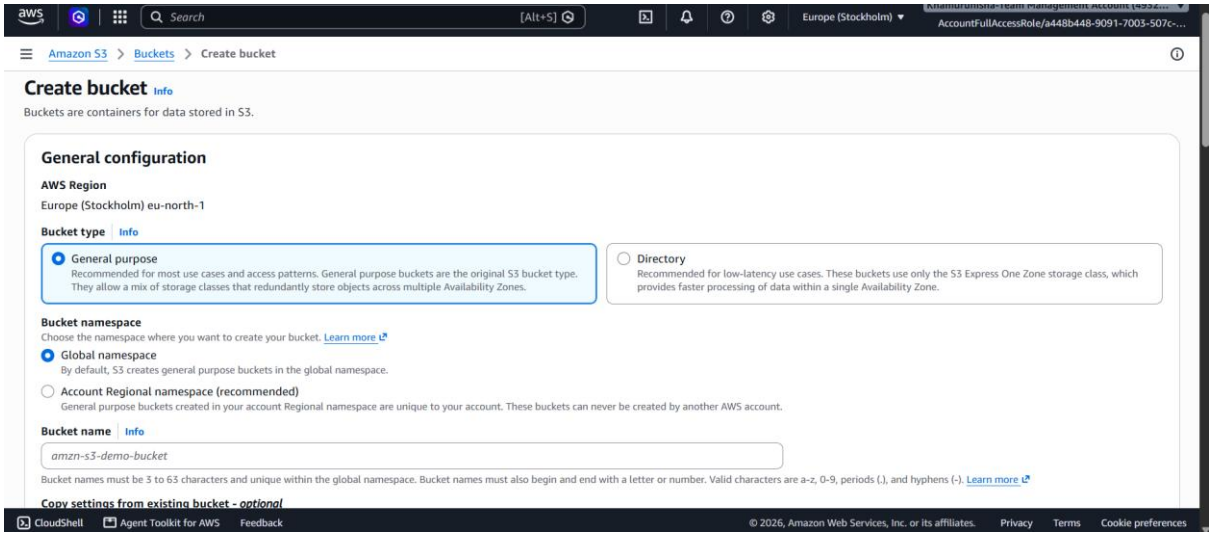


AWS Account / Region

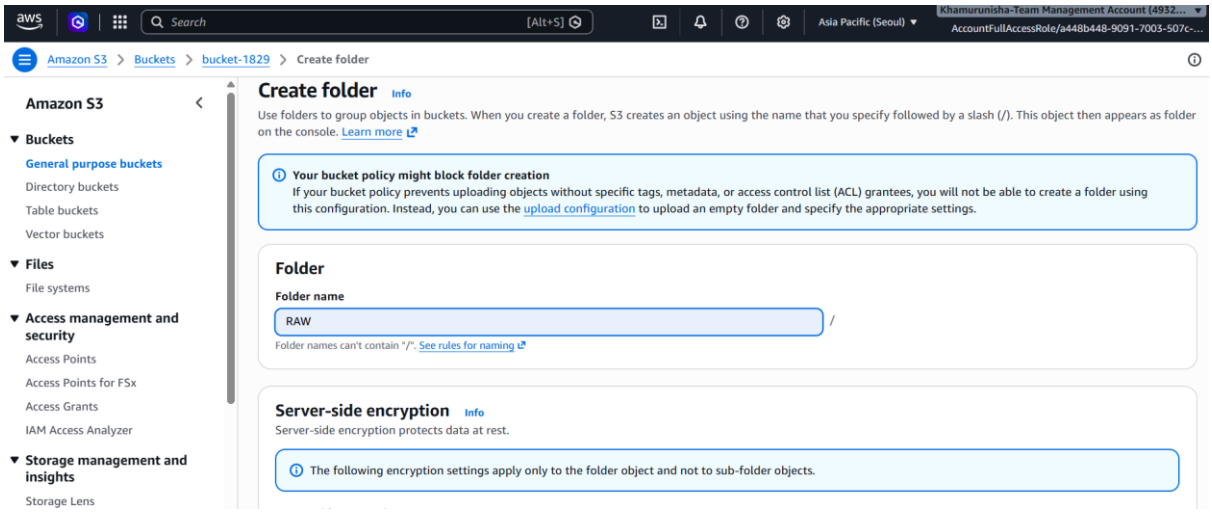


Amazon S3

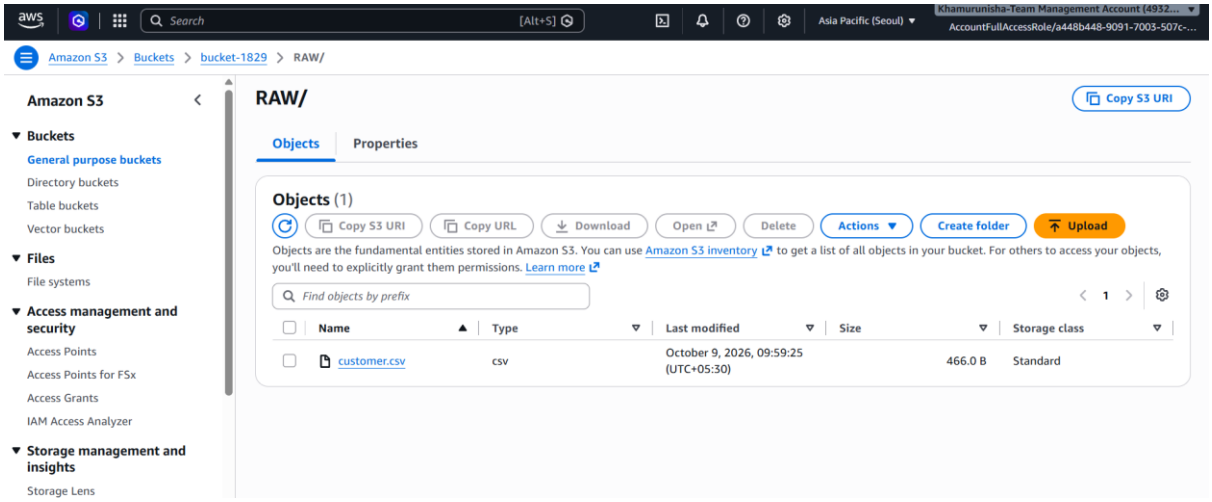
Create an S3 bucket



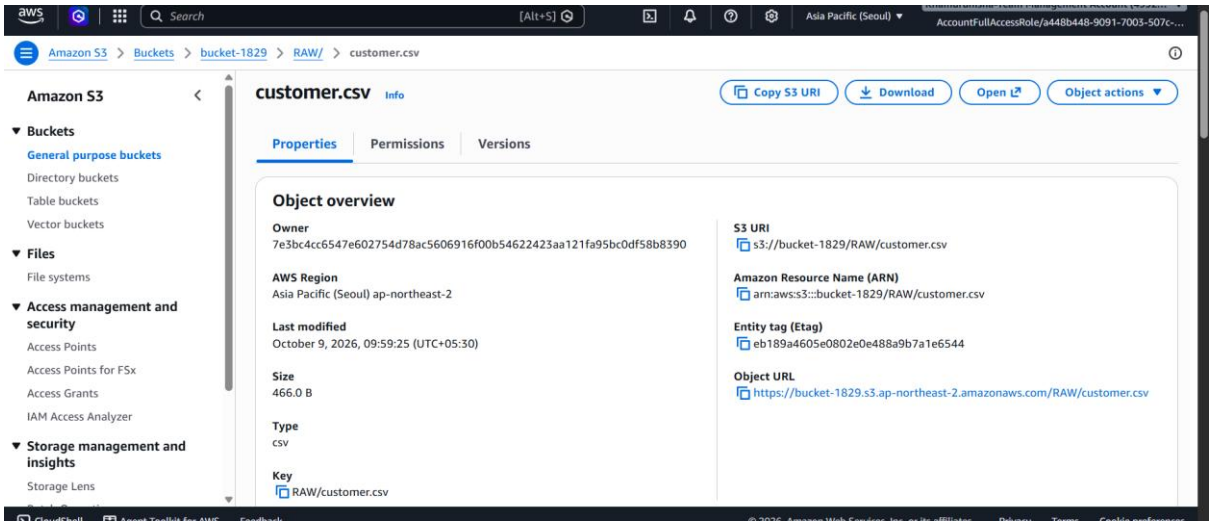
Create the raw/ folder



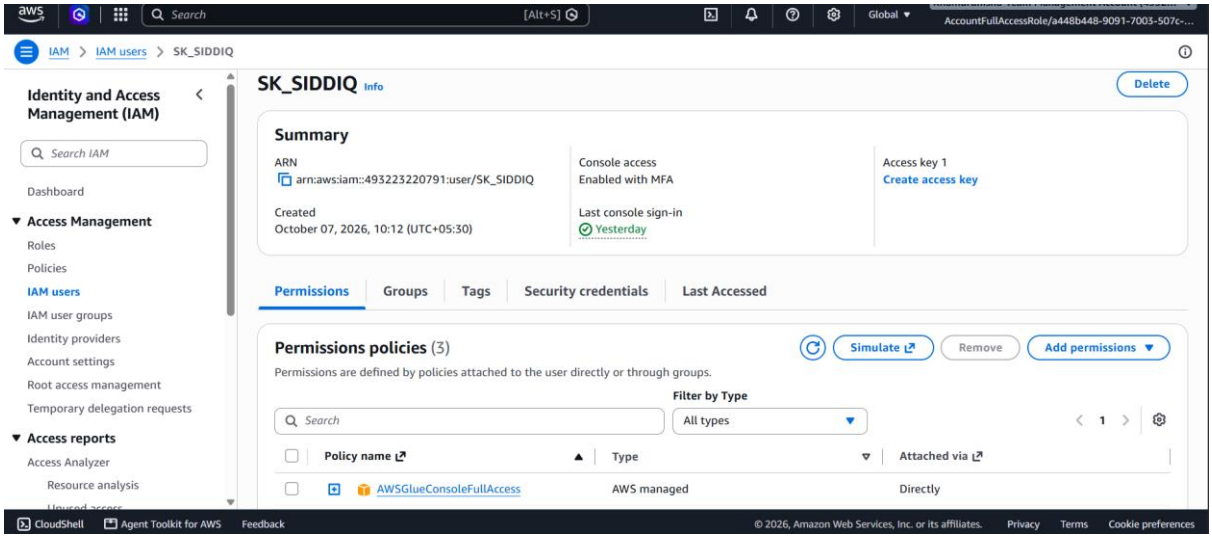
Upload the file into raw/



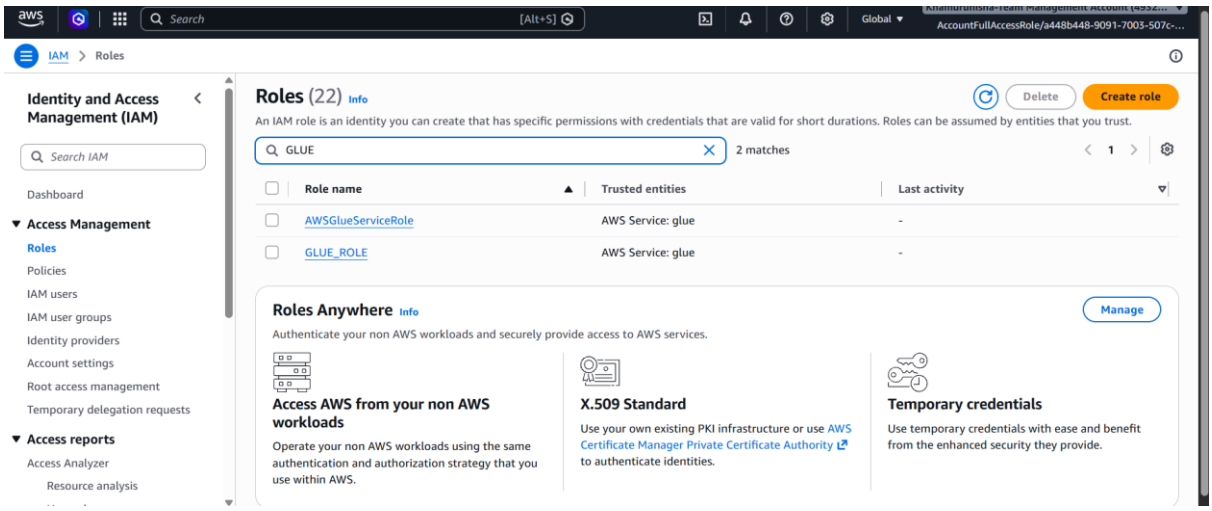
s3://bucket-1829/RAW/customer.csv



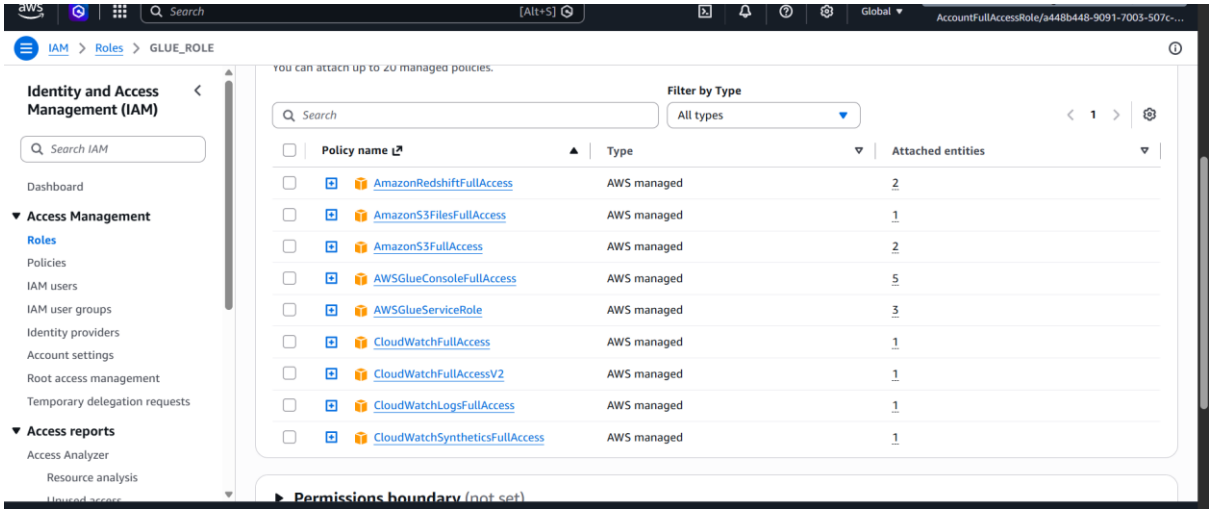
IAM(IDENTITY-ACCESS-MANAGEMENT)



GLUE-ROLE

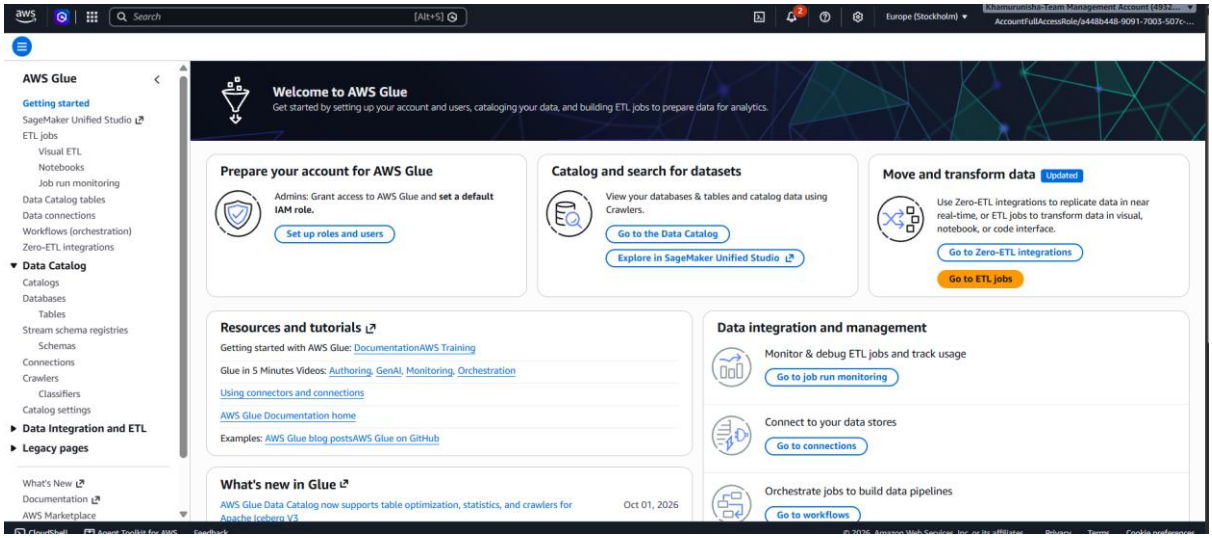


GLUE-ROLE-policies

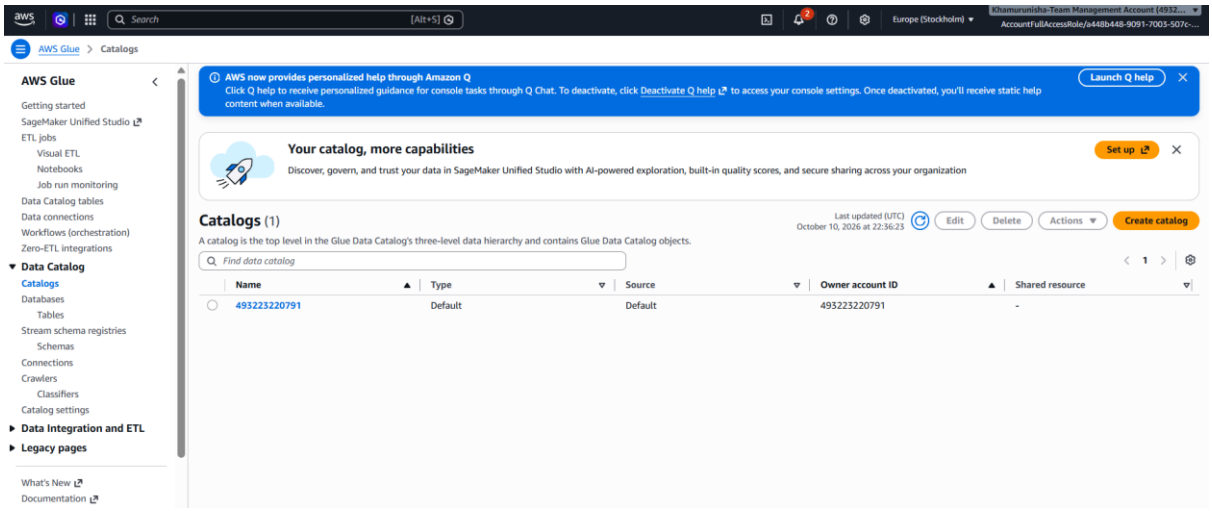


AWS Glue Database:

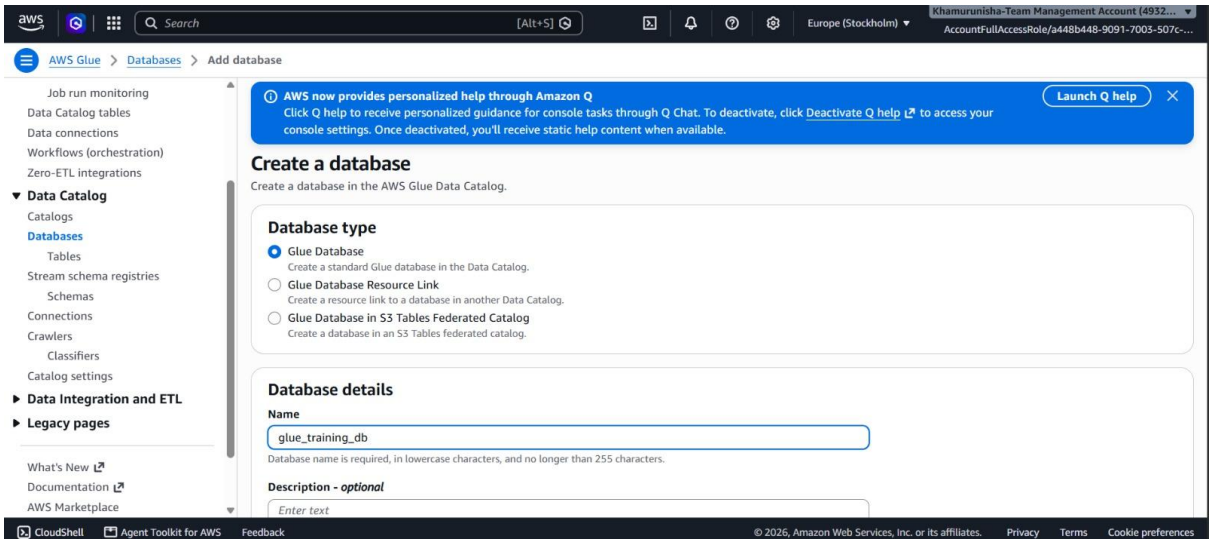
AWS Glue Console



Data Catalog

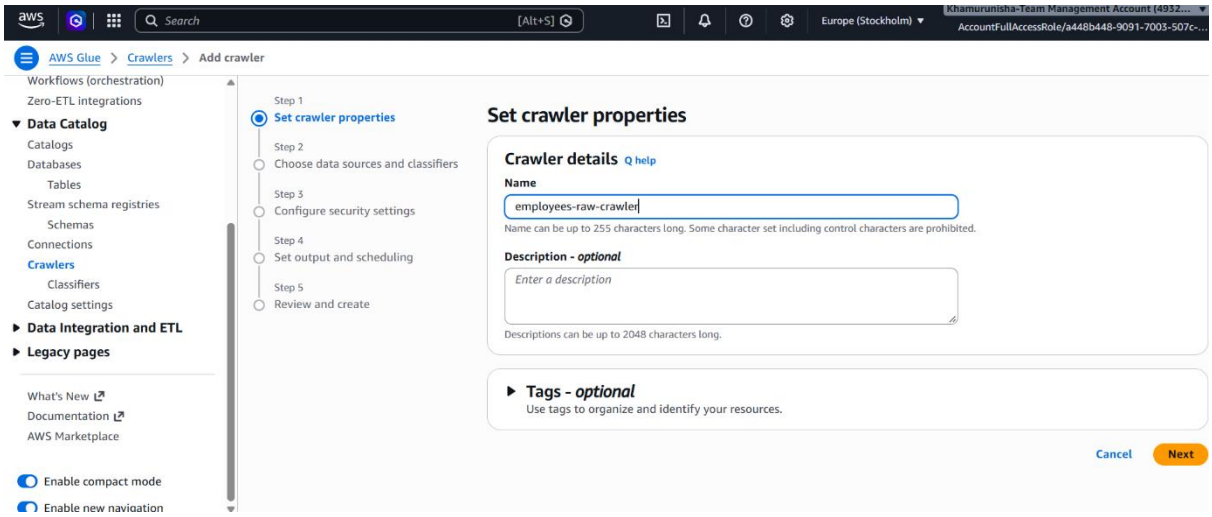


Database created

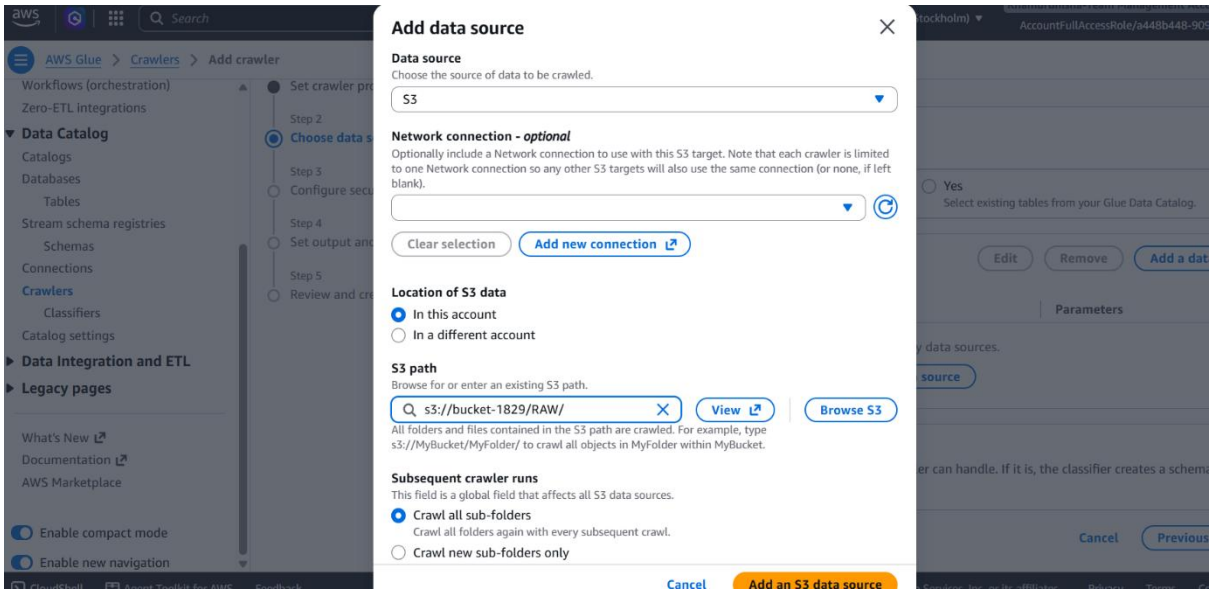


Glue Crawler

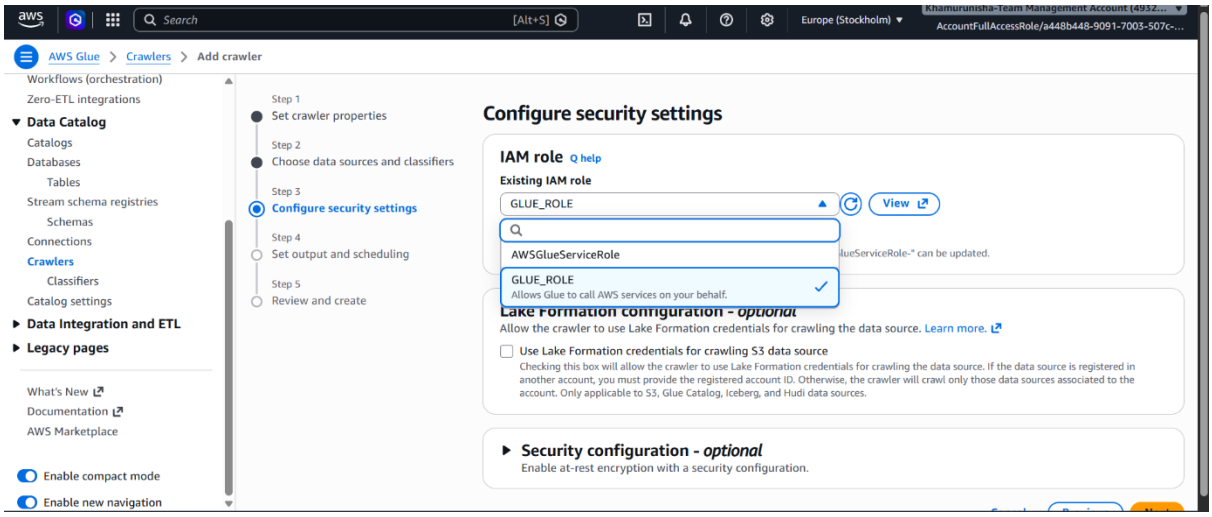
Crawler name



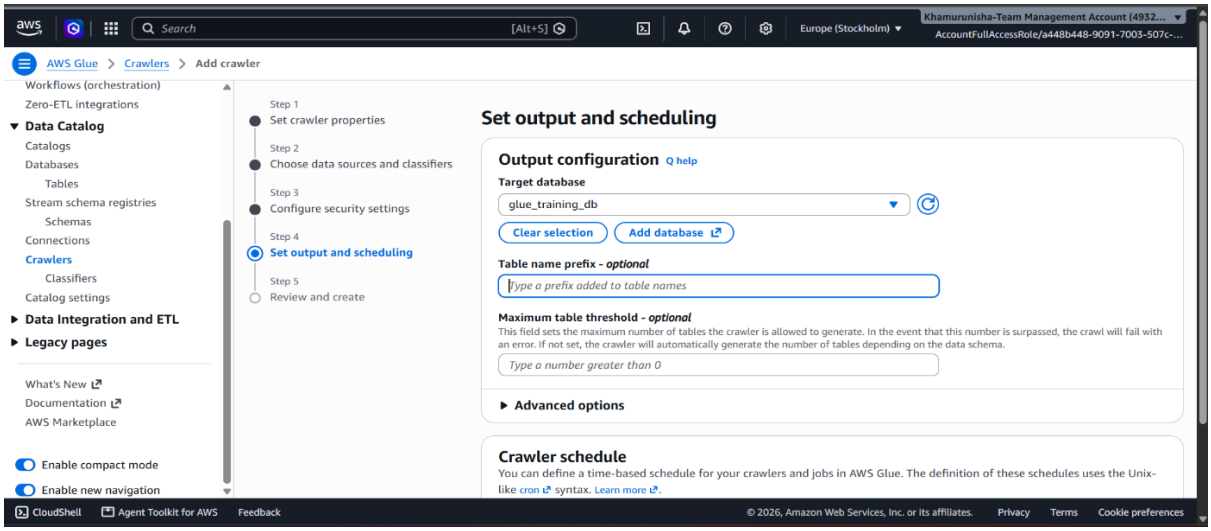
S3 source location



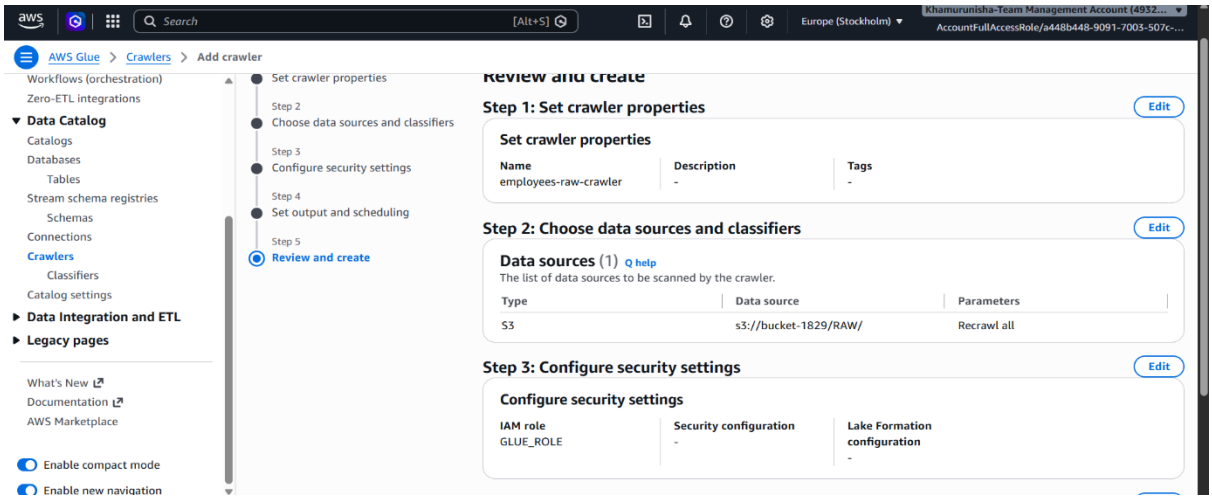
IAM role



Target database

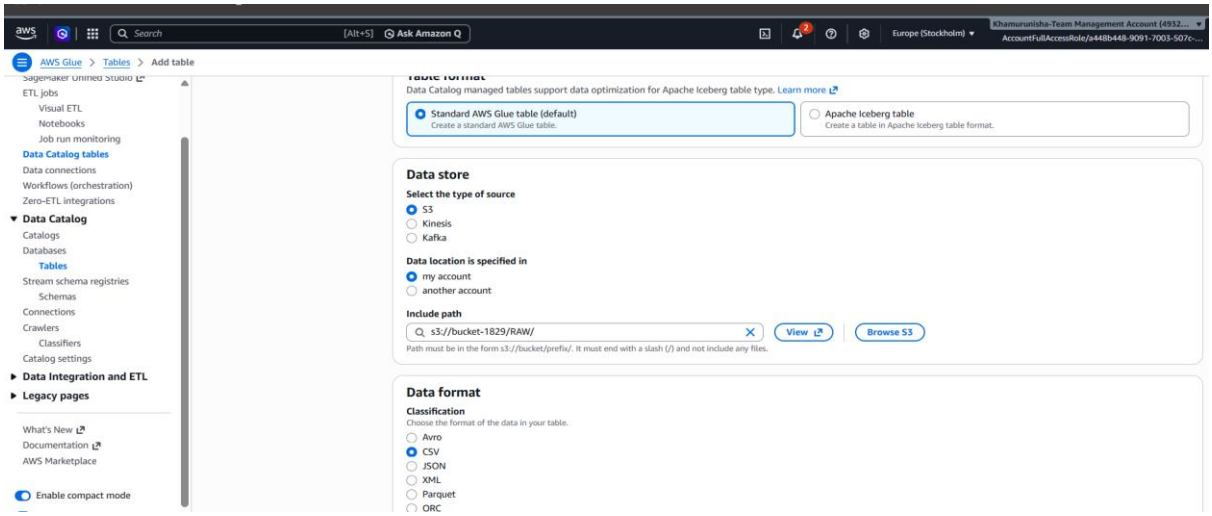


Review and create

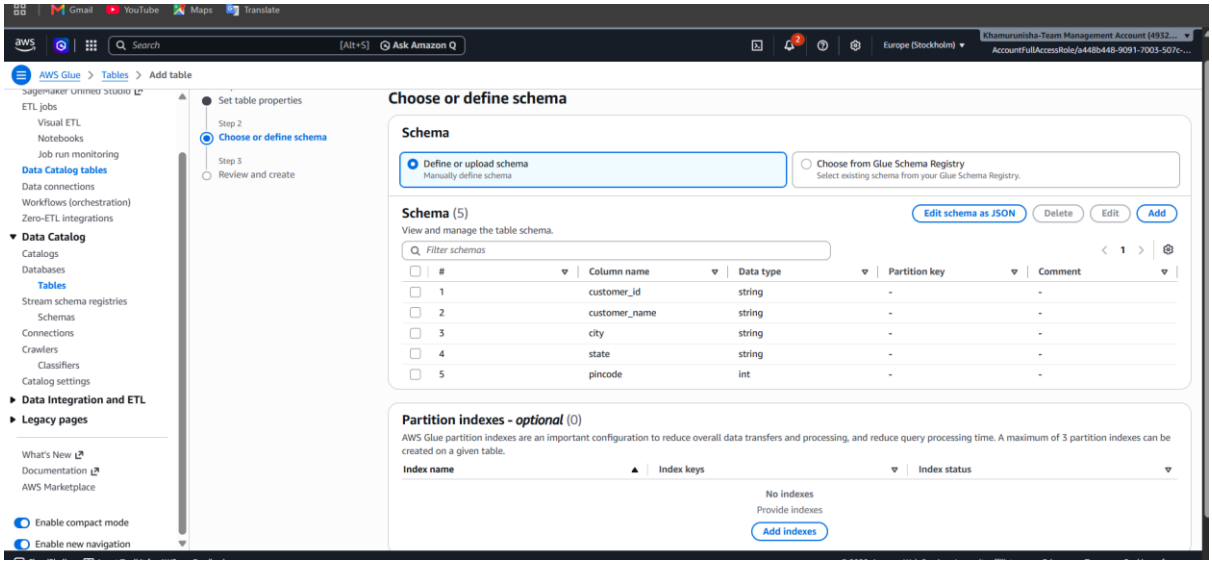


Glue Data Catalog

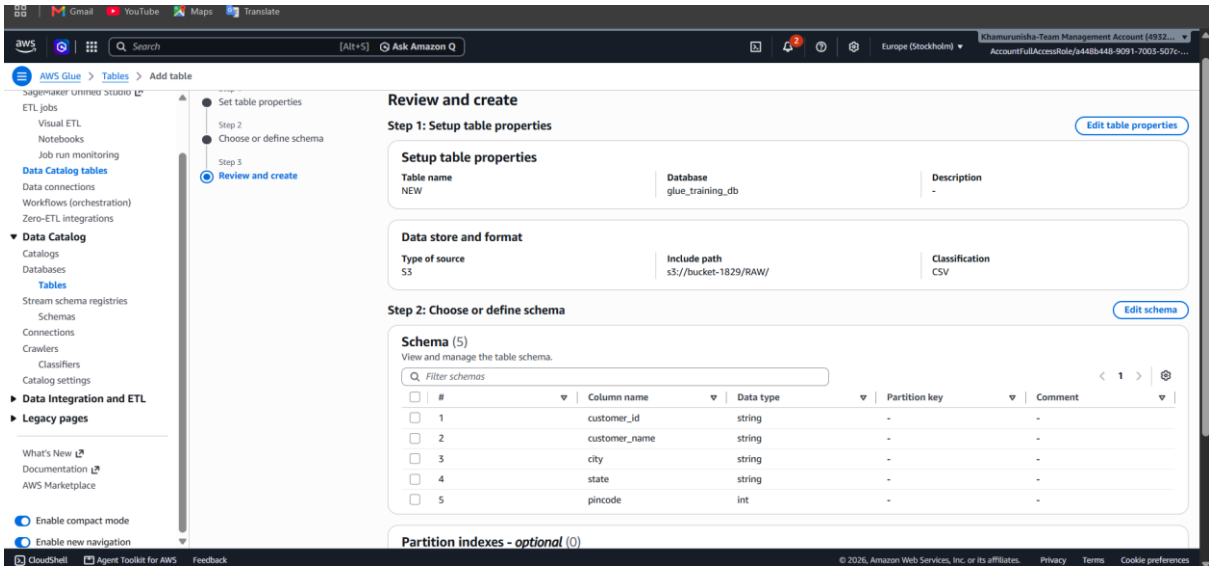
Create the table manually



Finish the schema

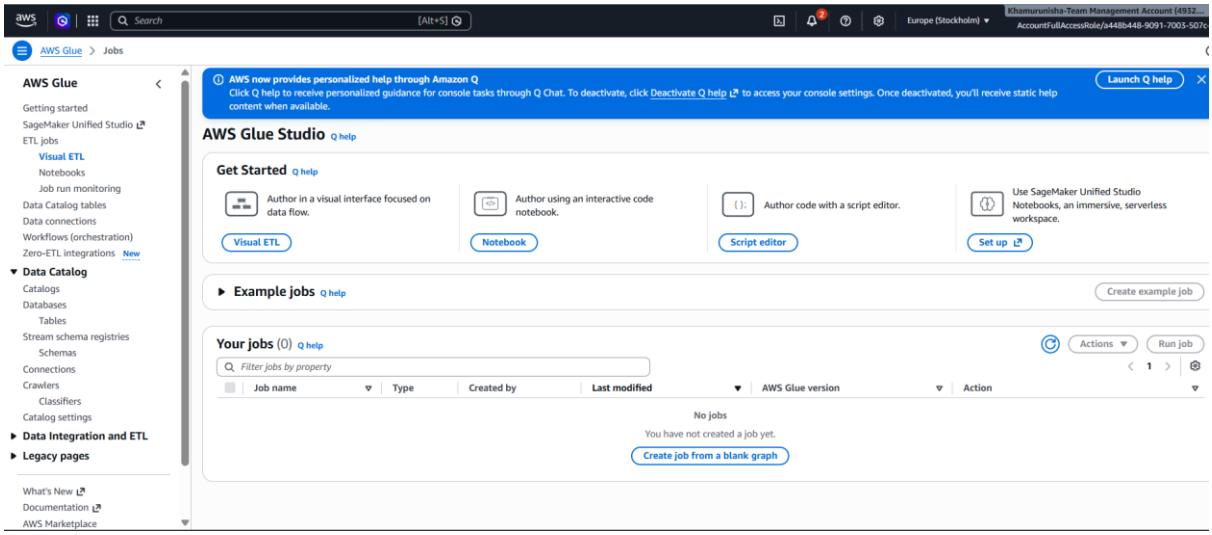


. Review and create

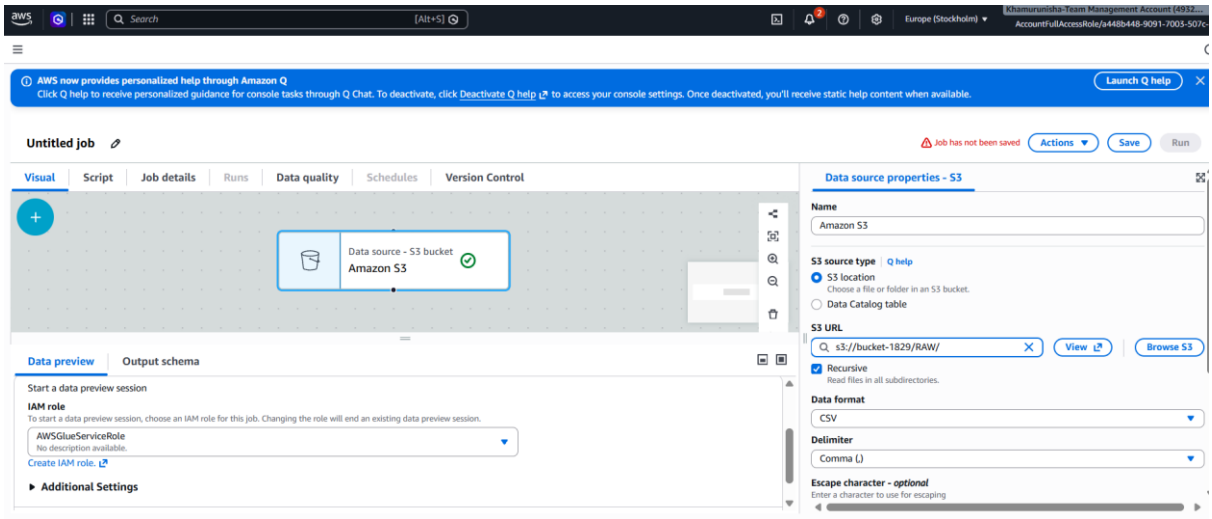


Glue Visual ETL

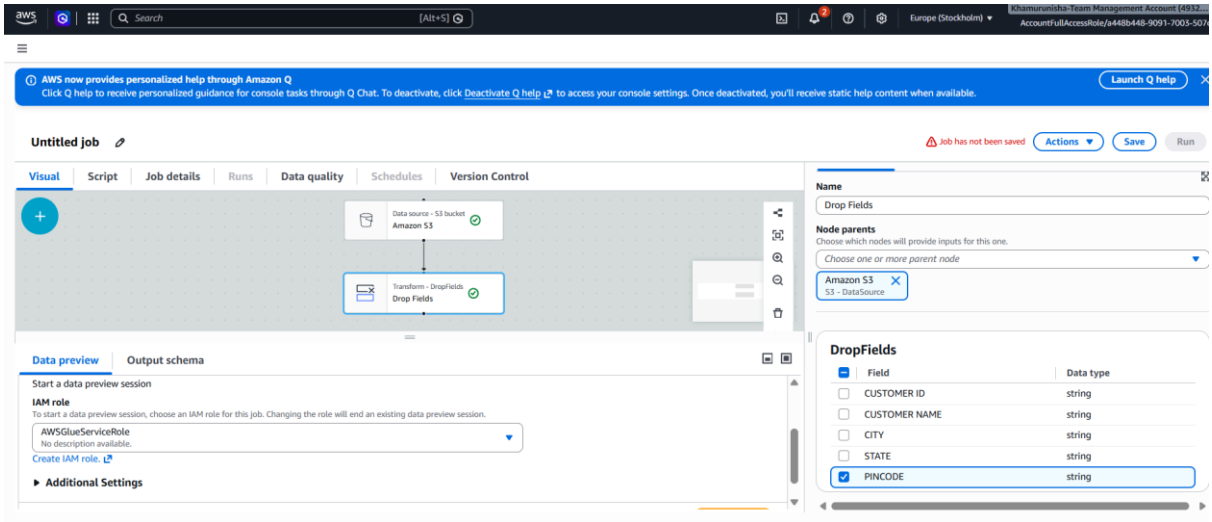
The complete workflow:



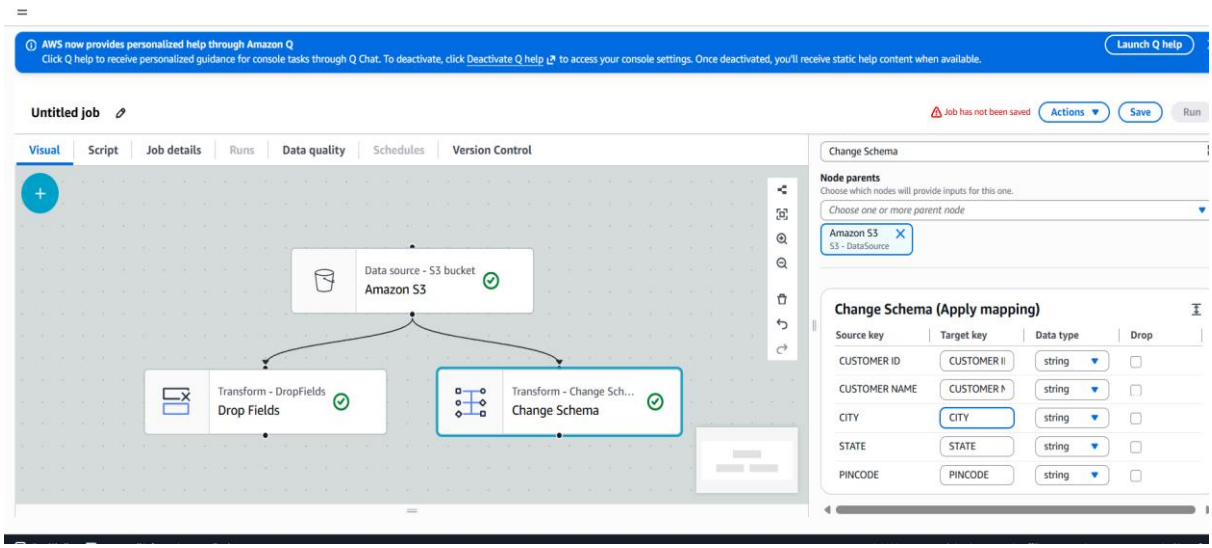
Create Source



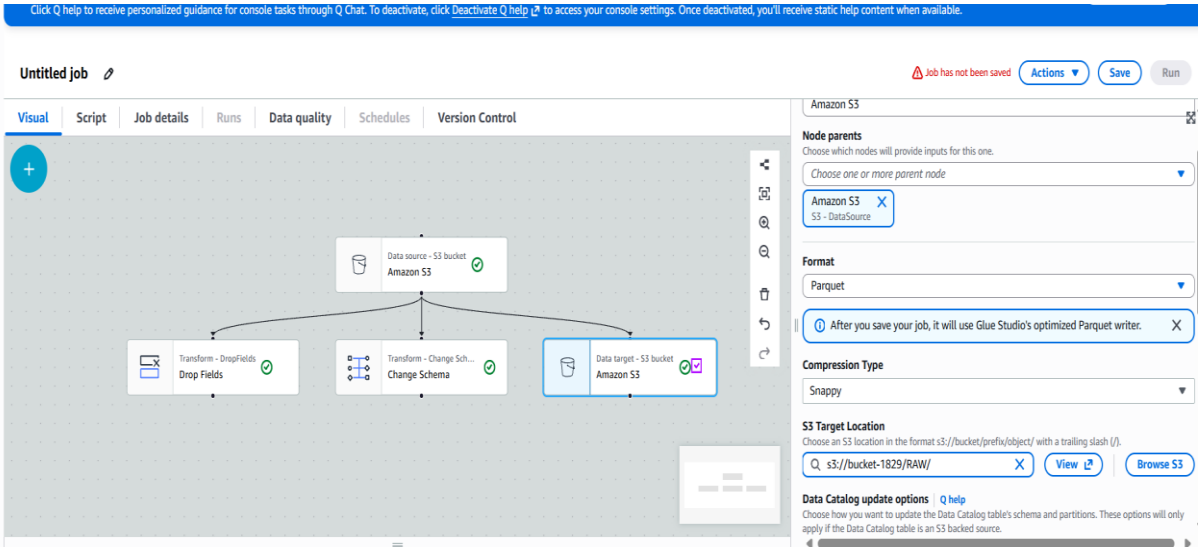
Glue Transformation(Drop Fields)



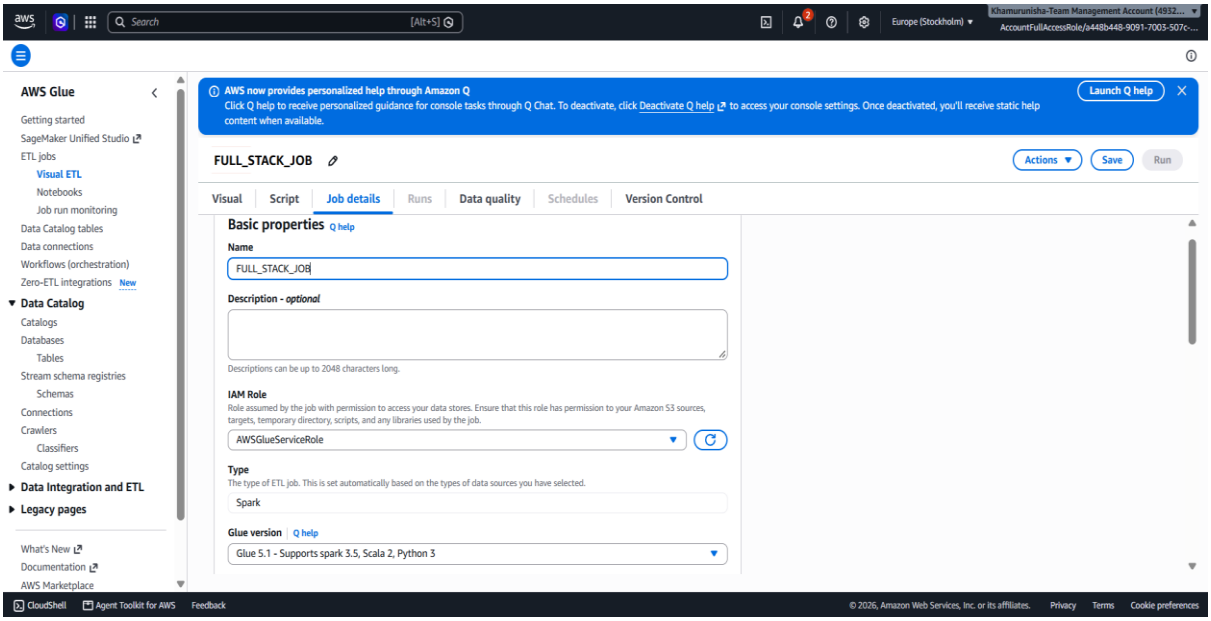
Glue Transformation



S3 Target Configuration



Glue ETL Job Execution



Final Explanation

Workflow: S3 Raw Data → Glue Crawler → Glue Data Catalog → Glue Visual ETL → Transformation → S3 Curated Data

Final Explanation

- **Amazon S3** (Raw Data): First, we upload the customer CSV file into the S3 bucket. It contains customer ID, name, city, state and pincode details.
- **AWS Glue Crawler**: The crawler scans the uploaded CSV file and finds the column names and their data types automatically.
- **Glue Data Catalog**: It stores the table information, column names and the location of the raw data. This helps Glue to understand and access the data easily.
- **AWS Glue Visual ETL**: In this step, we create an ETL job using the visual interface and connect the source data with the transformation and target location.
- **Transformation**: Here, we process the data according to our requirement. For ex, we can remove unwanted columns and keep only the required information.
- **Output Format**: After transformation, the data is saved in Parquet format, which helps to store and process data efficiently.
- **S3 Curated Data**: The final processed data is stored in the curated/ folder inside the S3 bucket.
- **Final Result**: Finally, the raw customer data is converted into a well organised format, which makes it easier to use for further analysis and reporting**.**

End-to-End Data Flow:

Customer CSV file → Amazon S3 (Raw Data) → AWS Glue Crawler → Glue Data Catalog → AWS Glue Visual ETL → Data Transformation → Convert to Parquat format → Amazon S3 (Curated Data) → Ready for analysis and report.